Freight Rate Prediction

Take-home assessment, Machine Learning Engineer (Remote), Spotter | Afsal Baaqir A | target: `posted_rate` (USD per load)

1. Summary

Problem. Predict the posted rate of 12,000 truckload loads in November-December 2025 from 48,000 labelled loads posted in January-October 2025, and produce a fixed December 2025 chart for one lane (Lexington → Fort Wayne, 360 mi, Dry Van, 32,000 lb).

What I found. The rate is mostly a lane level times a distance taper times an equipment premium. On top of that the data has deliberate traps: 1.41% of labels are multiplied by 0.17-0.47x or 2.1-5.4x (random corruption), `quote_signal` leaks the label with a sign that flips by month, `market_index` stops tracking prices from August, 8 validation cities never appear in training, coordinates are shifted, and weights contain sign flips, gaps and a cap.

Final model. An equal-weight average (in log space) of a tuned LightGBM (Huber loss, 7 leaves, 600 trees, recency weights with a 180-day half-life, 5 seeds) and a hierarchical shrinkage model (region pair → city → lane → lane+equipment, each level shrunk toward its parent). Both predict log(rate per mile). I picked it by a pre-defined selection score on time-based cross-validation.

0.648	**$43.5**	**$114.3**	**$813-832**
selection score (lane-median baseline 1.000; my previous version 0.764)	clean-row MAE (baseline $84.9; previous version $56.6)	unseen-city MAE (baseline $157.9; previous version $129.7)	December chart, mean $823 (lane's 21 loads in 2025: median $808)

Key limitations. Ten months of history contain no Thanksgiving or Christmas and no full year, so no holiday or yearly effect is modelled; all-row RMSE (about $626 for every model) is set by the random label spikes, which no model can predict; unseen-city error stays well above seen-lane error; `quote_signal` is unusable as given.

All numbers in this report come from one run of `python main.py` (14 minutes on my machine) and its `reports/` and `artifacts/` outputs, plus my data audit, the feature plots of section 2.1 and my first iteration's run where stated. Full tables are in the Appendix.

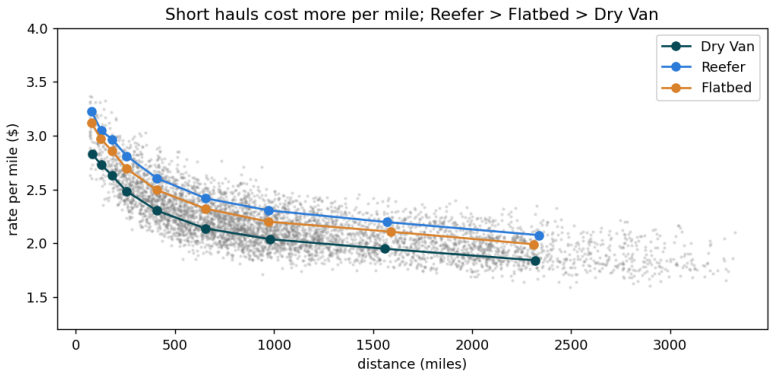
2. The data

file	size	content
`data/train_test.csv`	48,000 x 14	labelled loads, 2025-01-01 to 2025-10-31 (304 days), ids TR-000001.. sorted by date
`data/validation.csv`	12,000 x 13	same columns without `posted_rate`, 2025-11-01 to 2025-12-31 (61 days)
`data/validation_predictions_template.csv`	12,000 x 2	`load_id, predicted_rate` to fill
`data/december_chart_inputs.csv`	31 x 7	Lexington → Fort Wayne, 360 mi, Dry Van, 32,000 lb, one row per day of Dec 2025; no coordinates, market_index or quote_signal

Columns. `load_id`, `pickup`, `delivery`, `pickup_lat/lon`, `delivery_lat/lon`, `distance` (miles), `equipment` (Dry Van, Reefer, Flatbed), `weight` (lb), `date`, `market_index`, `quote_signal` and the target `posted_rate`.

Shape of the problem. Training has 64 cities and 4,014 directed lanes; validation has 72 cities. The median lane+equipment cell has only 3 loads and 64% of cells have fewer than 5, yet 84.6% of validation rows fall on a lane+equipment seen in training. Equipment mix is stable (train 56.7 / 25.1 / 18.2%, validation 56.5 / 25.4 / 18.1% Dry Van / Reefer / Flatbed). I found no exact or feature-level duplicates, no train-validation overlap, no pickup = delivery, no swapped coordinates and no spelling variants of cities or equipment.

What drives the rate. Rate per mile falls steeply with distance (median $2.95/mi up to 100 mi, $2.40 at 300-500 mi, $1.91 beyond 2,000 mi) and on the same lane Reefer is 13.0% and Flatbed 8.5% above Dry Van (figures 1-2). An OLS model on log rate per mile with distance, equipment, market_index, weight and a time trend reaches R^2 0.81 on clean rows, and adding origin and destination effects lifts it to 0.843: the lane level carries most of what is left.



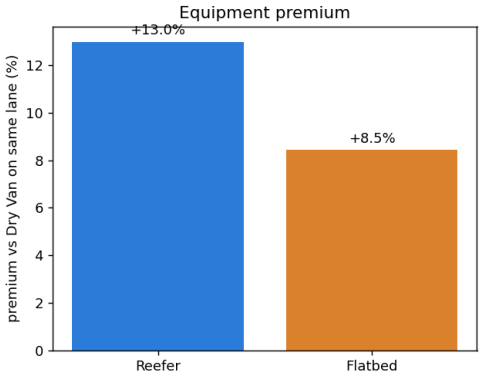


Figure 1 (left): rate per mile vs distance by equipment, clean rows. Figure 2 (right): equipment premium over Dry Van on the same lane.

2.1 How each feature relates to price

To see how each column moves the price before modelling, I plotted the labelled data directly (`report/feature_plots.py` , using the repo's own cleaning functions; the numbers quoted are in `report/figures_extra/feature_plots_stats.txt`). I show rate per mile next to posted_rate because distance dominates the raw rate. "Clean" means the 677 flagged spikes of section 3.1 are excluded; where they are shown they are red. Figures in this subsection are numbered x1-x6.

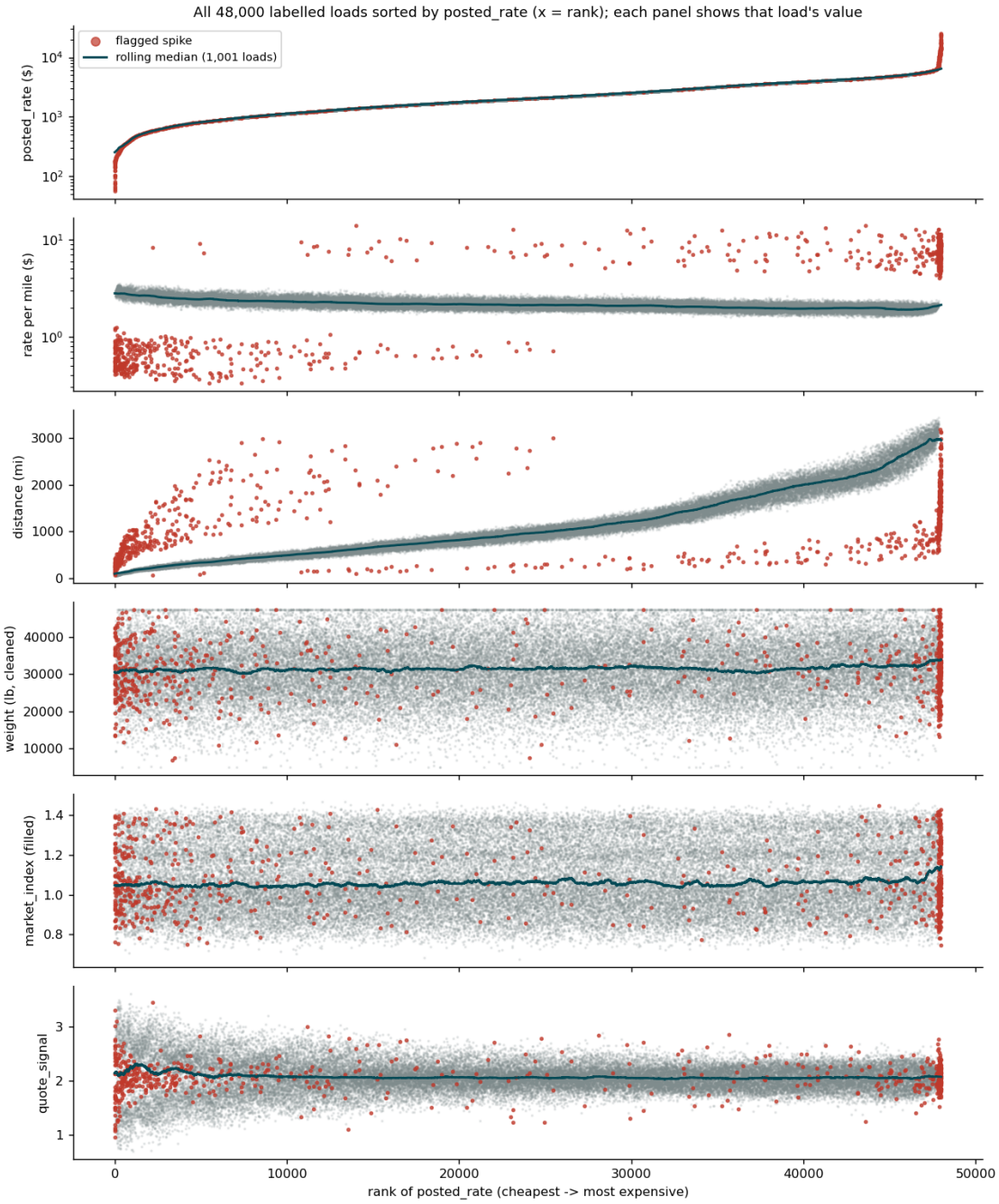


Figure x1: all 48,000 labelled loads sorted by posted_rate (x = rank, cheapest to most expensive). Each panel shows the same loads' rate per mile, distance, weight, market_index and quote_signal, with a rolling median over 1,001 neighbouring loads; flagged spikes in red.

Takeaway (x1). Sorting by price is almost the same as sorting by distance: the distance median climbs steadily from 237 mi in the cheapest decile to 2,556 mi in the most expensive, while rate per mile falls from $2.59 to $1.97. Weight (decile medians 30,963-32,601 lb), market_index (1.046-1.077) and quote_signal (2.04-2.18) barely change along the ranking, so none of them sets the price level on its own. The spikes collect at both ends: they make up 5.2% of the cheapest decile and 5.1% of the most expensive one, against 0.2-1.1% in the deciles between, and in the rate-per-mile panel they sit as two detached bands far from the clean cloud. A low-dip spike on a long haul lands among the cheap loads (red points above the distance trend on the left), which is why raw-price thresholds would not find them and a per-lane rule does.

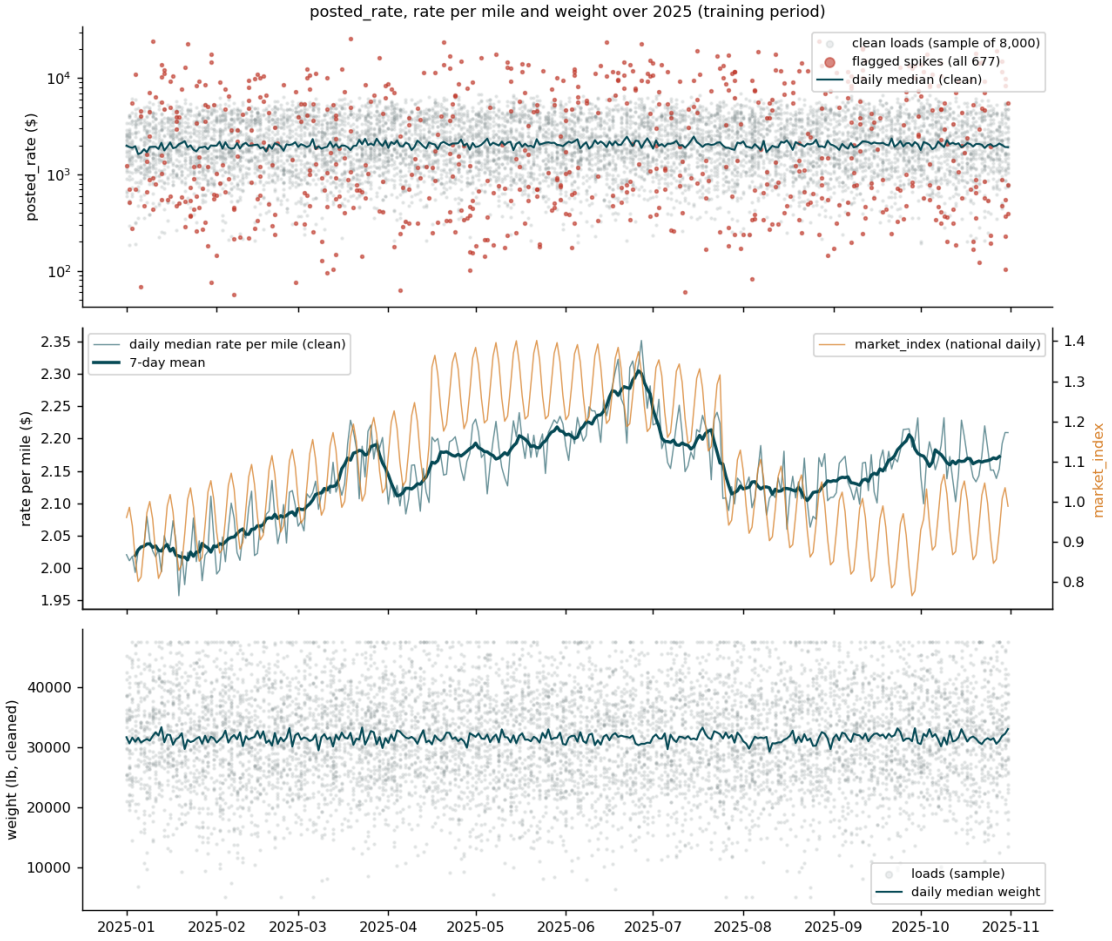


Figure x2: top, posted_rate per load over 2025 (sample of 8,000 clean loads, all 677 spikes, daily clean median; log scale); middle, daily median rate per mile with its 7-day mean and the national daily market_index; bottom, weight per load and its daily median.

Takeaway (x2). The daily median posted_rate is flat through the year because the distance mix does not change (monthly median distance 937-969 mi). Rate per mile does move: it climbs from about $2.02 in early January to a peak of about $2.30 at the end of June, drops in July and August and partly recovers in September-October (monthly medians $2.03 in January, $2.25 in June, $2.12 in August, $2.16 in October). market_index follows that path closely until July and then keeps falling while rates recover: the correlation between the daily median rate per mile and the daily index is 0.85 for January-July and 0.18 for August-October. This is the break that makes every market-level feature fail in section 5.2. The spikes are spread over the whole year rather than clustered in a period. Weight shows no trend over time (monthly medians 31,273-31,718 lb).

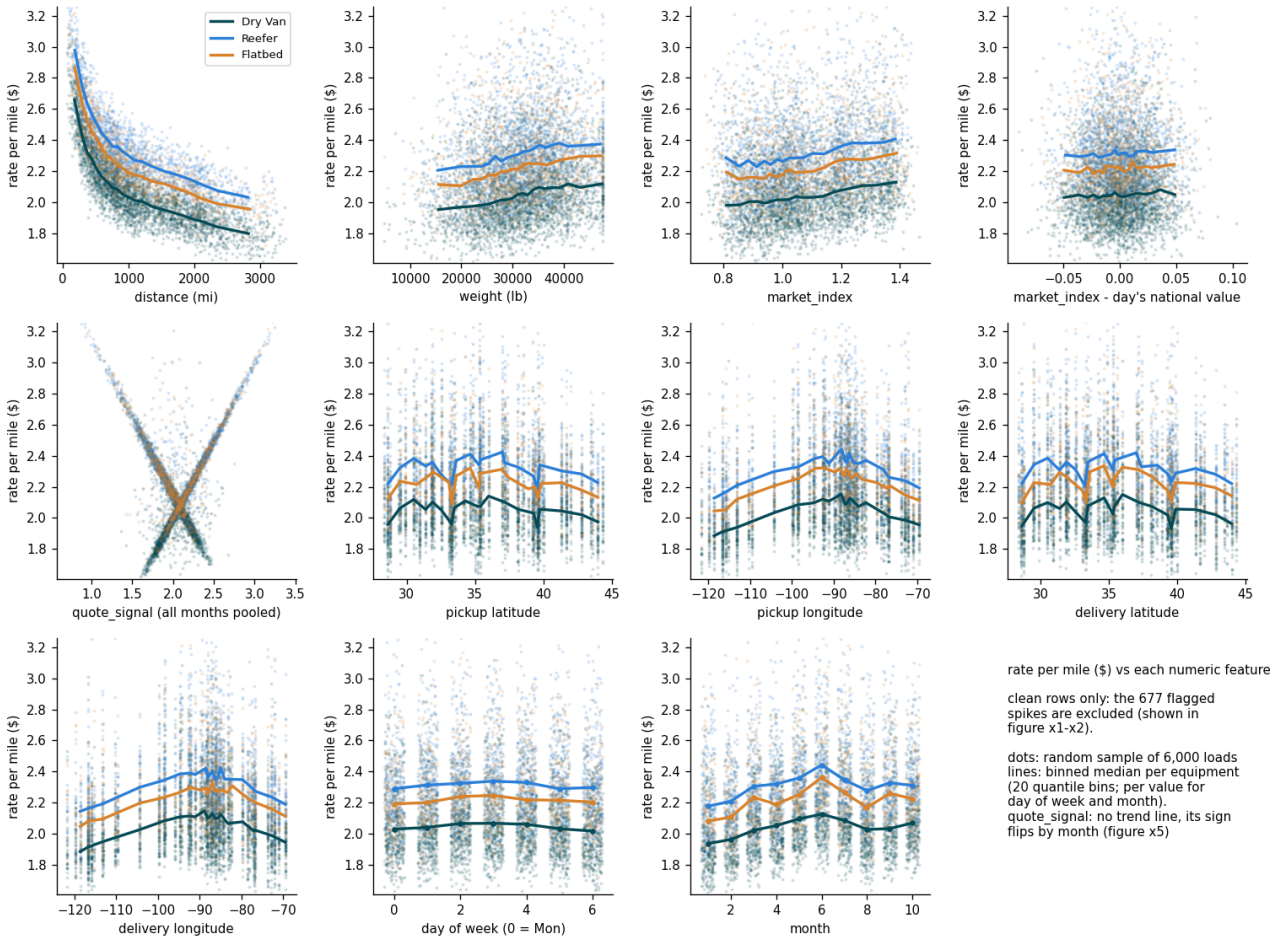


Figure x3: rate per mile against each numeric feature, clean rows. Dots: random sample of 6,000 loads, coloured by equipment; lines: median per equipment in 20 quantile bins (per value for day of week and month). quote_signal has no trend line because its sign flips by month (figure x5).

Takeaways (x3).

- **Distance** is the strongest driver: the Dry Van median falls from $2.66 to $1.80 per mile from the shortest to the longest bin, with the steepest drop below about 500 mi.
- **Equipment** shifts the whole curve: Reefer is above Flatbed, which is above Dry Van, at every distance, weight, index value, weekday and month. The gap is close to constant, which supports modelling it as a multiplicative premium.
- **Weight** raises the rate per mile steadily (Dry Van median $1.95 at about 15,500 lb to $2.12 at the 47,500 lb cap). Within the same lane and equipment, the lightest fifth of loads is priced 2.7% below the lane median and the heaviest fifth 2.4% above.
- **market_index**, pooled over the year, has a positive slope (within-lane: lowest fifth 1.9% below, highest fifth 2.7% above the lane median), but x2 shows that this slope comes from January-July and does not hold afterwards. The row deviation `mi_row_dev` has a small, positive slope (within-lane -0.3% to +0.4% across its fifths).
- **quote_signal** pooled over months forms two mirrored lines (an X) instead of a cloud: a sign of a constructed column rather than a market quantity.
- **Coordinates**: rate per mile is lowest for western pickups and deliveries (longitude about -120) and highest around -90 to -85, falling again toward the east coast; latitude shows no clear trend. These are city-level differences that the city and lane encodings capture more directly.
- **Day of week and month**: a small midweek hump (within-lane mean from 0.992 of the lane median on Sunday to 1.009 on Thursday) and the seasonal level seen in x2 (within-lane mean 0.960 in January, 1.036 in June).

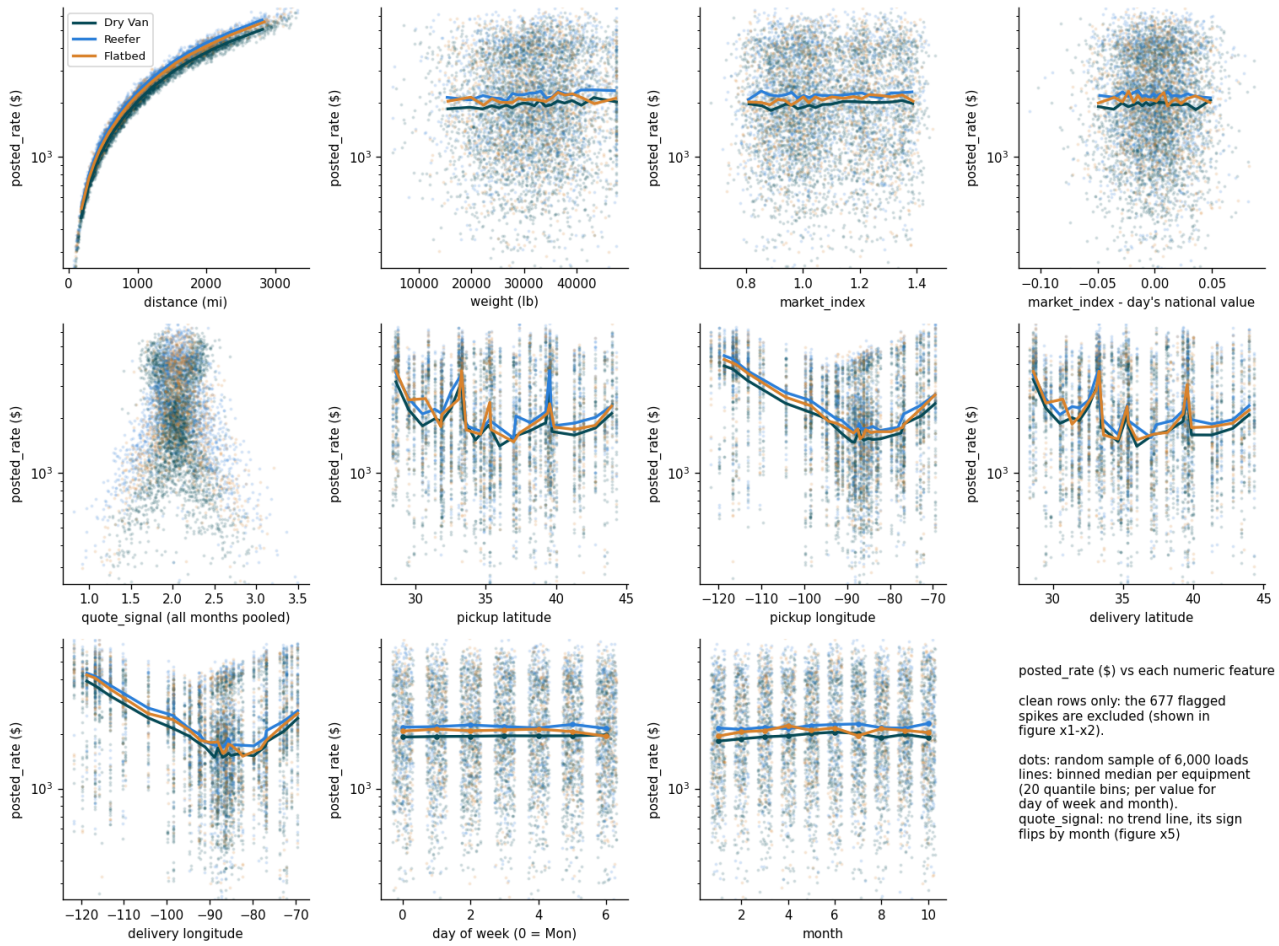


Figure x4: posted_rate (log scale) against the same features, clean rows; same layout as x3.

Takeaway (x4). In raw dollars distance explains almost everything (Pearson 0.989 on clean rows) and the other features look flat, because their effects of a few percent are hidden by distances that range from 70 to over 3,000 miles. The apparent price differences by longitude and latitude are mostly distance mix (pickup longitude correlates -0.27 with distance): western cities have longer hauls. This is why I model rate per mile, not the raw rate.

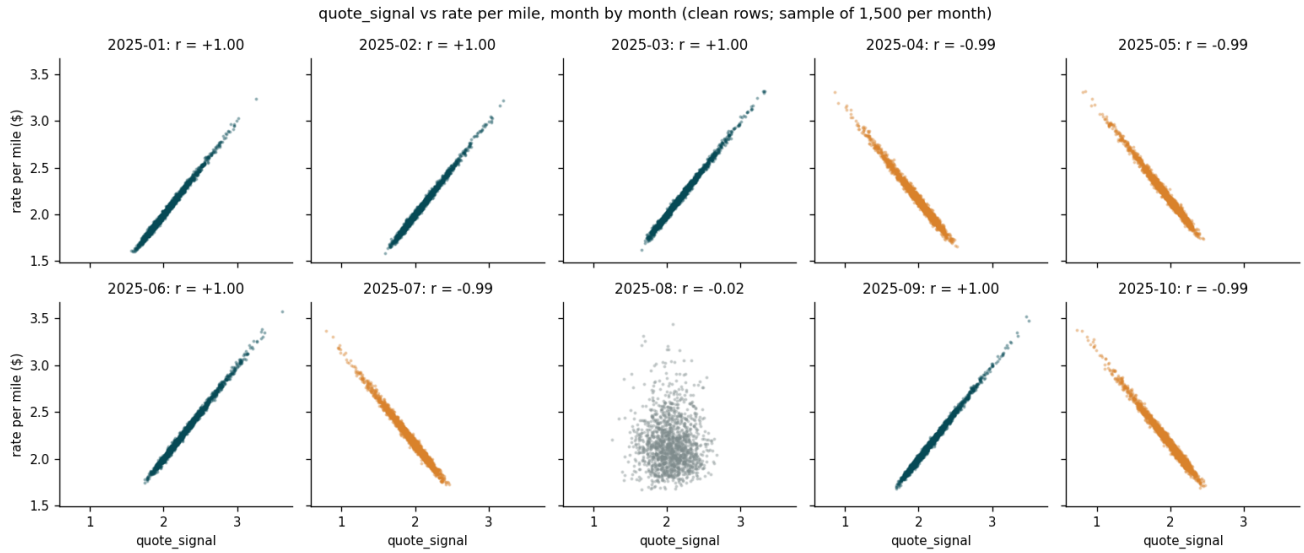


Figure x5: quote_signal against rate per mile, one panel per training month (clean rows, 1,500 sampled loads per month); r = Pearson correlation over all clean loads of the month.

Takeaway (x5). Month by month quote_signal is an almost exact linear copy of the rate per mile: r = +1.00 in January, February, March, June and September, -0.99 in April, May, July and October, and -0.02 (no relation) in August. A

column that reproduces the label this precisely, with a sign that changes from month to month and vanishes in validation (section 3.2), cannot be used for prediction.

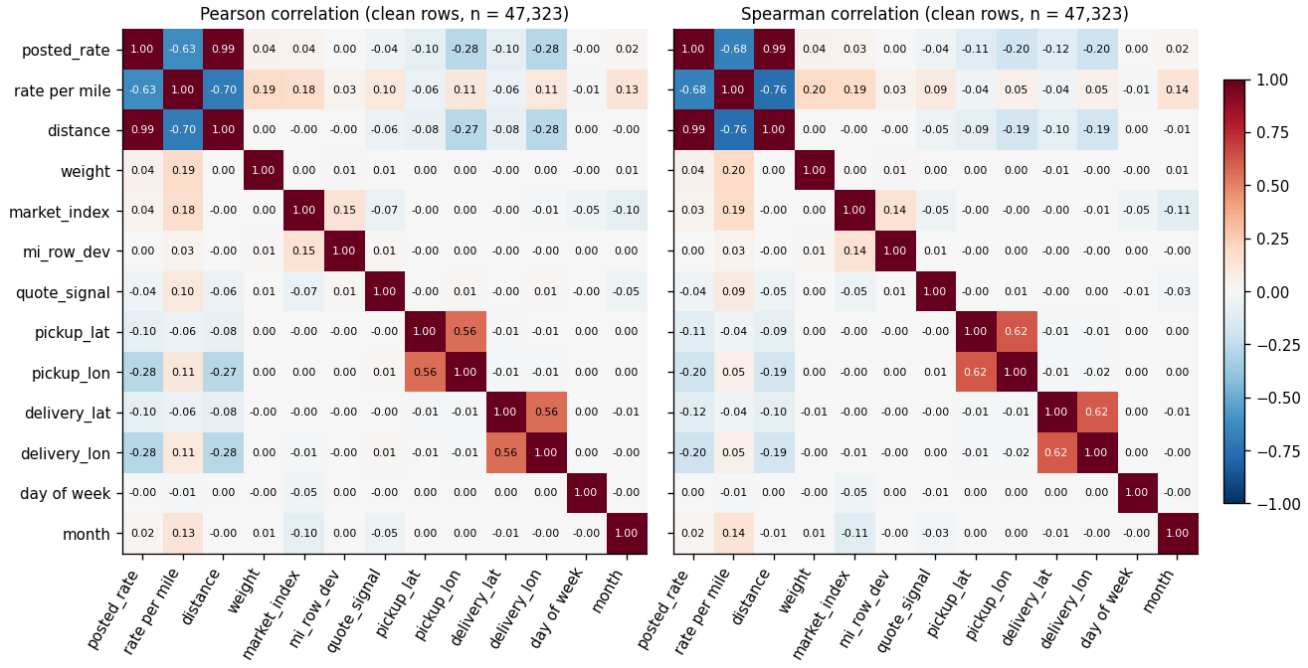
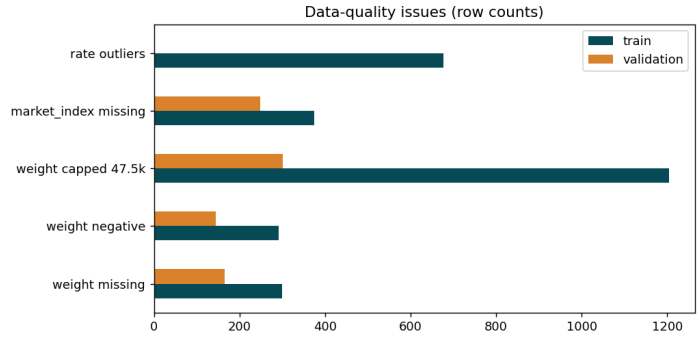


Figure x6: Pearson (left) and Spearman (right) correlations between the numeric columns, posted_rate and rate per mile, clean rows (n = 47,323).

Takeaway (x6). For rate per mile the ranking is distance (Pearson -0.70, Spearman -0.76), weight (0.19 / 0.20), market_index (0.18 / 0.19), month (0.13 / 0.14) and quote_signal (0.10 / 0.09, meaningless because its sign flips); day of week and mi_row_dev are near 0 in linear terms although their within-lane effects above are real but small. Pearson and Spearman agree, so the relations are monotone rather than driven by a few points. The spikes matter here too: with them kept, the correlation of rate per mile with distance drops from -0.70 to -0.34 and with weight from 0.19 to 0.09. Correlations cannot show equipment, which is categorical (figure 2) and whose removal costs the most in the refit ablation of section 5 (clean MAE 44.0 to 106.9), nor the lane level, which I handle with encodings.

3. Data issues and how I handled each

issue	size	handling (details below)
`posted_rate` spikes	677 train rows (1.41%)	flag per lane; drop from training only; Huber loss
`quote_signal` leak	every row; sign flips by month	excluded as a feature
`weight` gaps / sign flips / cap	465 missing, 437 negative, 1,505 at 47,500 lb (train + validation)	abs(); equipment-median fill; clip to 5,000-47,500; 3 flags
`market_index` missing	623 (374 train, 249 validation)	fill with the day's national value; use only the row deviation
cities absent from training	8 cities, 1,447 validation rows (12.1%)	borrow encodings from 3 nearest known cities; cold-start CV folds
shifted coordinates	up to ~5.7° per city	trust `distance`; coordinates only as coarse location
holidays only in validation	Thanksgiving, Christmas	no holiday effect modelled or invented



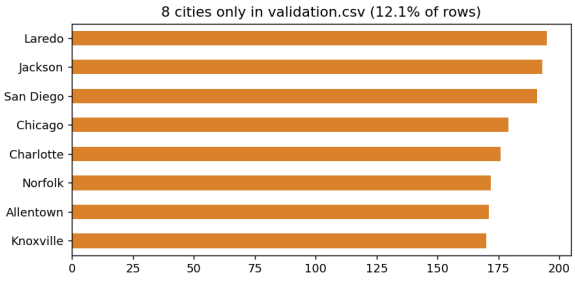


Figure 3 (left): row counts of each data-quality issue in train and validation. Figure 4 (right): validation rows per city that never appears in training.

3.1 posted_rate spikes (677 rows, 1.41%)

Evidence. I compared each load's rate per mile with the median of its lane+equipment (or, when the lane has fewer than 3 loads, the median of its equipment and distance band). Clean rows are very tight around that reference (standard deviation of the log ratio 0.042), but a separate population sits far away: 340 high spikes at 2.08-5.44x and 337 low dips at 0.17-0.47x (figure 5). The gaps between them are empty: no load between 0.5x and 0.8x and one between 1.25x and 2.0x, while 47,322 loads lie within 0.8-1.25x. Real market moves would fill those gaps continuously; a detached cluster at whole multipliers points to corrupted prices. None sit near 10x or 0.1x, so they are not unit or decimal errors that could be undone.

Are they predictable? I tested this before deciding to drop them (`reports/outlier_diagnostics.md` , Appendix A6). High and low are balanced (binomial p = 0.939). Across 24 factors (equipment, distance band, cities, regions, lane size, month, weekday, holidays, market_index, weight flags, quote_signal, load order) no flag rate differs after Holm correction. Two classifiers trained out of time on 29 features cannot find them: pooled AUC 0.508 [0.482, 0.534] for logistic regression and 0.513 [0.489, 0.537] for LightGBM. Daily counts behave like a constant rate (dispersion 0.98, p = 0.569) and lane clustering is within what a random-corruption simulation produces (p = 0.075; no lane significant after Holm).

Decision rule and reasoning. I pre-set the rule that a factor explains the spikes only if its Holm-adjusted p < 0.01 and a classifier's AUC interval lies above 0.55. Nothing passed, so I treat the spikes as random label noise. I flag rows outside 0.6x-1.6x of the reference (clean rows sit within about ±5%, so these bounds leave a wide margin) and drop them from training only; inside CV the flag is recomputed on each fold's training part. Validation rows are never altered: every one gets a prediction of its clean rate, which is the best possible guess for an unpredictable multiplier. Measured effect of keeping them in training: clean MAE 44.5 vs 44.0. I still train with a Huber loss in log space because it limits the pull of any noise the rule misses; in my first iteration, keeping the spikes and switching to squared-error loss raised forward-fold clean MAE by $31.1.

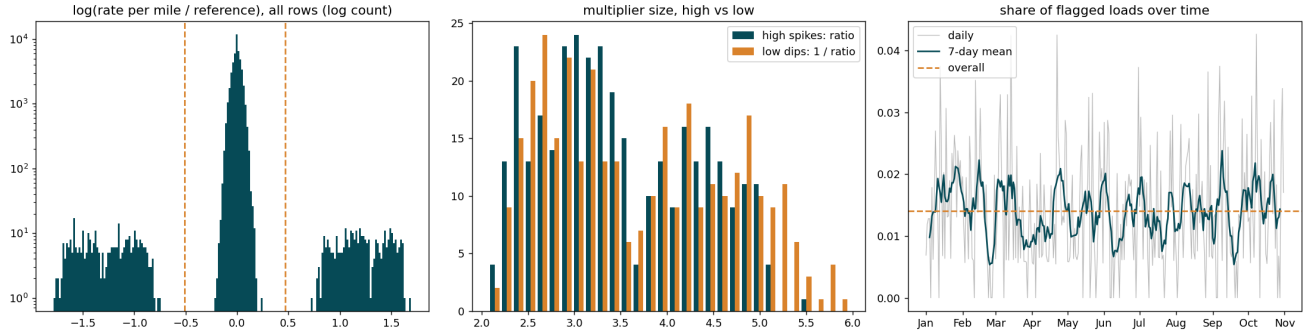


Figure 5: left, log ratio of rate per mile to the lane reference (log count) with the 0.6x / 1.6x bounds; middle, multiplier sizes of high spikes and low dips (as 1/ratio); right, daily share of flagged loads over time (flat around 1.4%). Figure 10 in `reports/figures/` shows the flag rate by each factor with 95% intervals.

3.2 quote_signal leaks the label, with a month-dependent sign

Across the whole year `quote_signal` looks like noise (correlation with the rate -0.04). Month by month it is not: its correlation with the lane rate per mile is +0.95 to +0.97 in Jan, Feb, Mar, Jun and Sep, -0.96 in Apr, May, Jul and Oct,

about 0 in August, and 0.00 / -0.02 in validation November / December (figure 6, Appendix A5). Within a month it is essentially the rate per mile (or its mirror image) plus small noise, so it was built from the label. I excluded it for three reasons: the sign cannot be known for a new month, it is pure noise in the months I must predict, and a model that uses it learns a rule that breaks whenever the sign flips. Measured: adding it raises clean MAE from 44.0 to 48.0 (it helps on some folds and fails on rolling_08, where August is noise).

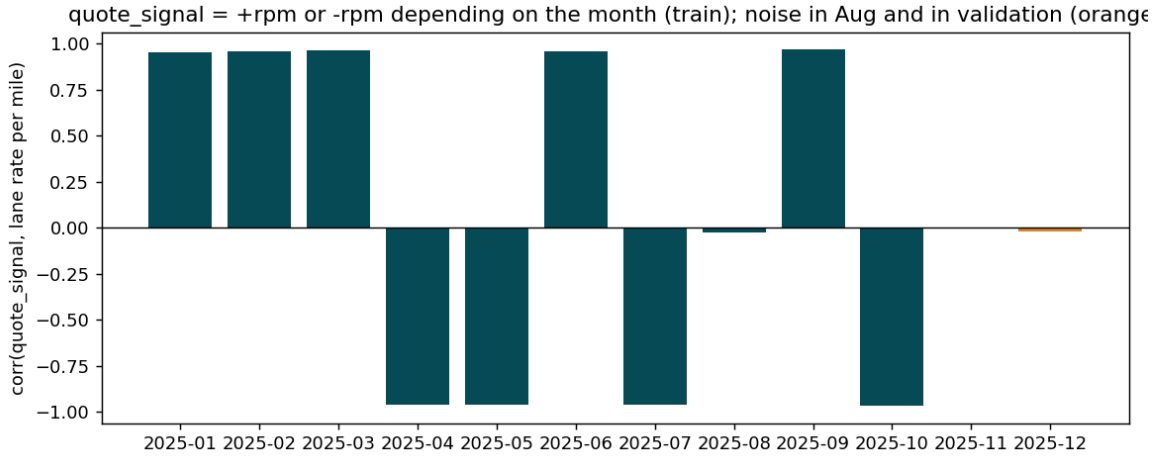


Figure 6: correlation of quote_signal with the lane rate per mile, by month. Dark = training months; Nov-Dec (validation, labels unknown) uses the lane median from training and is near zero.

3.3 weight: missing values, sign flips and a cap

- **Negative values (437)** are sign flips: their absolute values have the same distribution as positive weights (median 31.8k vs 31.5k lb) and the same weight-to-rate gradient, so I take `abs()`.
- **47,500 lb** is a pile-up (2.5% of training rows sit exactly there), i.e. a cap at the legal limit; 5,000 lb is a floor. I clip to [5,000, 47,500] and flag capped rows.
- **Missing values (465)** are spread randomly across months and equipment. I fill them with the training median weight of the same equipment type. The median, because the cap pile-up and the floor would pull a mean; per equipment, because equipment is known for every row and is the natural group for a trailer's typical load. Weight's effect is modest (about +3% per 10,000 lb), so the imputation error costs little, and a flag lets the model treat filled rows differently.

Three flags are kept (`weight_missing`, `weight_was_negative`, `weight_capped`). Weight matters as a group: removing it raises clean MAE from 44.0 to 65.5.

3.4 market_index: one national value per day

All regions correlate 1.0 day by day, so `market_index` is one national daily series plus row noise (row std 0.025 vs between-day std 0.17). The daily mean imputes missing values with R^2 0.978, so I fill the 623 gaps with that day's national value (computed from the train and validation files, which also covers every December chart date) and flag them. The series has a seasonal hump (monthly mean 0.93 in January, 1.30 in May, 0.89 in September) and a strong 7-day cycle, high on Thursday and low on Sunday / Monday (figure 7). Section 5.2 explains why I use only the row-level deviation from the daily value, not the level itself.

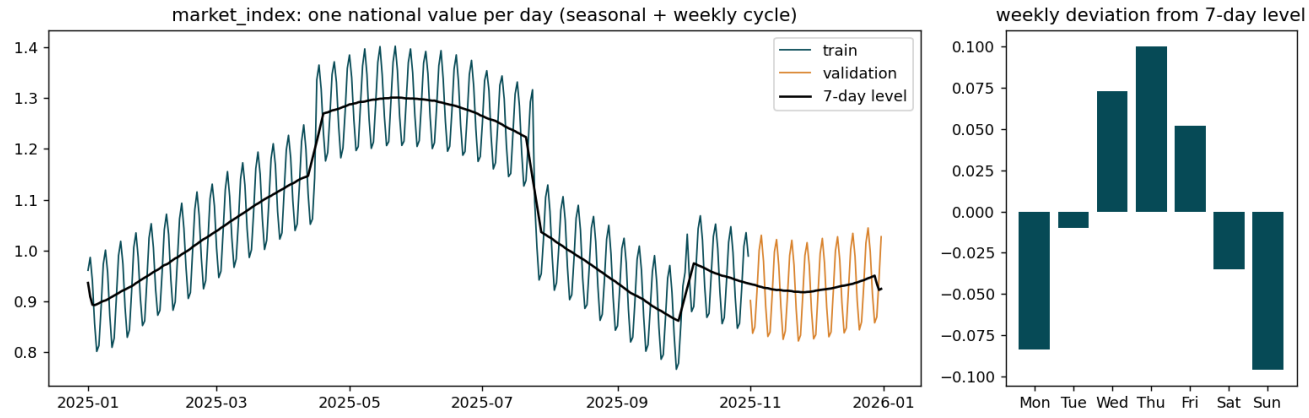


Figure 7: daily market_index (train and validation) with its 7-day level, and the average weekly deviation by weekday.

3.5 Eight cities that only appear in validation

Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk and San Diego cover 1,447 of 12,000 validation rows (12.1%; never both ends of the same load). One-hot or plain target encoding has nothing to say about them. I give each new city the inverse-distance-weighted encoding of its 3 nearest known cities (haversine distance between the given coordinates, weights 1/d) and add `pickup_seen` / `delivery_seen` flags; lane encodings are left blank, so trees take their learned default branch and the hierarchical model falls back to region and city effects. I chose haversine over squared degrees because a degree of longitude is about 20% shorter than a degree of latitude at US latitudes, and k = 3 so that one badly placed neighbour cannot dominate. I measure this case directly with cold-start folds (section 4).

3.6 Shifted coordinates

Each city has exactly one coordinate pair, consistent across files, but shifted from its real position by up to about 5.7° (Los Angeles sits at 28.57, -116.70; Allentown lands 0.9 mi from New York). As a result the ratio of `distance` to the haversine distance has a median of 1.18 and ranges up to 9.6 in training and 78.5 in validation. The `distance` column itself is consistent within a lane (coefficient of variation 1.7%), so I trust it and use coordinates only as coarse location: raw latitude / longitude, their differences, four longitude bands as regions, and for choosing nearest neighbours. I did not "repair" distances from coordinates.

3.7 Holidays are absent from training

Thanksgiving and Christmas occur only in validation. The training holidays (New Year, Memorial Day, 4 July, Labor Day) show no effect beyond what market_index carries, and the spike rate within 3 days of a holiday equals the overall rate (1.41%). With no history to learn from, I model no holiday effect rather than invent one.

4. Target and validation design

Target: log(posted_rate / distance). Rate per mile removes most of the distance effect; the log turns the multiplicative structure (lane level x distance taper x equipment premium x market level) into a sum that trees and linear models handle well, and the label noise is multiplicative too. Predictions are converted back with exp(.) x distance and floored at $150 (the smallest clean training rate is about $185). A model of log rate estimates a median-like value; section 6.4 tests correcting that toward the mean.

Why not random K-fold. The data are ordered by date and validation is the two months after training. Random folds would put loads from the same day (same national market level) and the same lane on both sides and reward memorising the date, which is impossible for Nov-Dec. Every fold therefore trains on the past and tests on the future:

- **Rolling monthly folds** rolling_07 ... rolling_10: train on all months before M, test on month M.
- **Forward fold**: train Jan-Aug, test Sep-Oct, a two-month horizon like the real task.
- **Cold-start folds** (3 repeats): like the forward fold, but 8 random cities and every training load touching them are removed, mimicking the 8 new validation cities. Rows touching those cities give the *unseen-city* metrics.

Inside every fold the outlier filter, target encodings and hierarchical effects are fitted on that fold's training part only, and target encodings for training rows are 5-fold out-of-fold, so no row sees its own label.

Metrics. MAE, RMSE and MAPE in dollars on three subsets: *all* test rows, *clean* test rows (spikes excluded, using a flag from all labelled data, for scoring only) and *unseen-city* rows. Spotter did not name its metric, so I chose a selection score that balances them, fixed before comparing models: **selection score = mean(all-row MAE, clean-row RMSE, unseen-city MAE), each divided by the lane-median baseline**; lower is better and the baseline scores 1.000. All-row RMSE is reported but not used: it is about $626 for every model because the spikes dominate it.

5. Features

5.1 Features I introduced (the shipped set)

group	features	why	clean MAE without it
distance	`distance`, `log_distance`	short hauls cost more per mile (figure 1); the target already divides by distance, so this models the taper	45.5
equipment	`equipment_code`	Reefer +13.0%, Flatbed +8.5% over Dry Van	106.9
weight	`weight_clean` + 3 flags	heavier loads price higher; flags mark repaired values	65.5
mi_row_dev	`mi_row_dev`, `market_index_missing`	row value minus the day's national value: the only market signal that holds in every month	44.5
day of week	`dayofweek`	weekday pricing pattern learned from rates, not from the index level	46.7
geo	lat / lon, d_lat, d_lon, pickup / delivery region	coarse location; needed to place unseen cities	44.3
city encodings	`te_pickup`, `te_delivery`, seen flags	origin / destination level; the vehicle for unseen cities	44.0
lane encodings	`te_lane`, `te_lane_equipment`	lane level beyond what its two cities explain	44.6

Last column: tuned LightGBM refit on the 5 time folds without that group (shipped recipe: clean MAE 44.0). Full table in Appendix A4.

Target encodings. I encode the residual of a tiny baseline (log rate per mile on log distance and equipment), not the raw target, so a city does not look expensive just because its loads are short. Each group mean is shrunk toward 0 with a pseudo-count of 20, so a lane with 3 loads mostly keeps the prior. City encodings and geo look redundant on these time folds because every test city is known there; they matter for unseen cities, which only the cold-start folds contain.

Time level: recency weighting (chosen). The price level drifts a few percent during the year (clean rows vs each lane's mean: -4.1% in January, +3.6% at the June peak, -0.8% in August, +0.7% in October; Appendix A5). Rather than a time feature, I weight each training row by $0.5^{\text{age / 180 days}}$, so recent months count more. It only re-weights real history and cannot extrapolate a trend that has already turned. It beat pooling (clean MAE 44.0 vs 45.1); the half-life was tuned (section 6.2).

5.2 Features I considered and rejected

Every option below was tested the same way: the tuned LightGBM refit on the 5 time folds with one change (single seed). Full numbers, including per-fold values, are in Appendix A4.

option	clean MAE	why it was rejected
shipped recipe (recency weights, half-life 180)	44.0	-
pooled level (no time handling)	45.1	slightly worse than recency weights
linear trend, extrapolated / held at last day	111.9 / 94.7	keeps climbing or falling after the level has turned; held, it still starts from the line's overshooting end point
trailing 30- / 60-day level	64.2 / 60.9	carries the latest level forward; over-predicts after the June peak (rolling_07 bias +4.9%)
season (month + day of year)	61.2	trees cannot extrapolate past the last training date and repeat the last months' level
market_index level (4 columns)	91.4	relation to price breaks from August (below)
lags 1 / 7 / 14 days	91.0 / 84.5 / 81.7	inherit the same break
leads 1 / 7 / 14 days	92.1 / 99.1 / 85.4	same break; also unknown in production
7-day change, 28-day ratio	68.2 / 61.2	same break, smaller
market level x equipment	92.6	same break
quote_signal	48.0	label leak with month-dependent sign (section 3.2)

Why every market_index level feature hurts. Until July the monthly index moves with the price level; from August the index keeps falling (0.89 in September) while prices recover (+0.4% in September, +0.7% in October) (figure 8). A model trained on earlier months learns "low index = cheap" and under-prices Sep-Oct. Lags, leads, changes, ratios and the equipment interaction are all functions of the same series, so they inherit the break. What does hold in every month is the row deviation `mi_row_dev`: a load whose index is above that day's national value is priced slightly higher. Removing it raises clean MAE from 44.0 to 44.5, so I kept it as the only market feature.

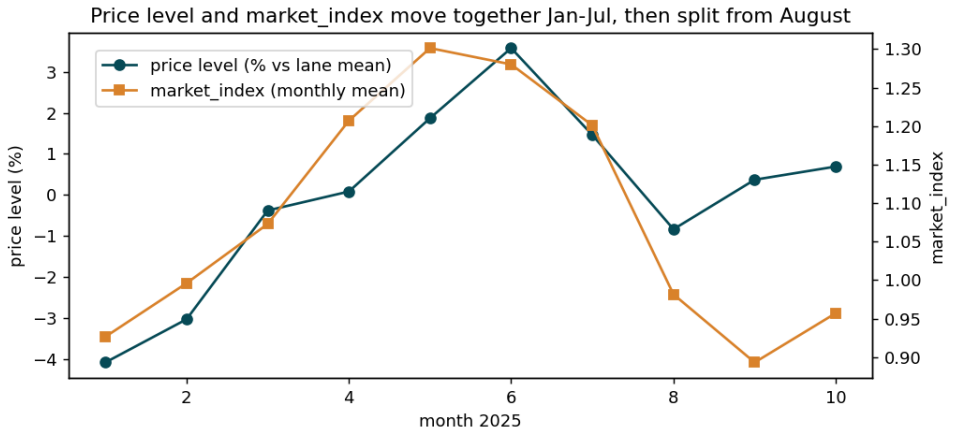


Figure 8: monthly price level (% vs lane mean, clean rows) and monthly mean market_index. They move together January-July and split from August.

Day of week (kept). Removing it raises clean MAE from 44.0 to 46.7. It is also what gives the December chart its weekly shape (section 8).

Hierarchical encodings and city masking (first iteration). In my first iteration I also fed LightGBM hierarchical encodings (each level shrunk toward its parent) and trained with city masking (12% of training rows pretend their city is unseen so the model learns to use borrowed encodings). In that run they gave small gains over plain LightGBM: selection score 1.088 → 1.079 (hierarchical encodings), 1.087 (masking alone) and 1.076 (both); unseen-city MAE 193.5 → 189.7 / 192.6 / 190.0 (Appendix A7). The gain of about $1-3 was of the same size as the spread between the three cold-start city sets, so in the final version I dropped both from the LightGBM features and used the hierarchy as a separate, transparent model in the blend instead, which helps more (section 6).

Why I replaced permutation importance. My first report had a contradiction: refitting without season raised forward-fold clean MAE by about $61, yet season's permutation importance was exactly 0. Both were correct. In the forward

fold every test date lies after the last training date, so the trees send all test rows down the same "latest months" branch and shuffling month among them changes nothing. Refitting without season, however, left the market-level features in charge, and those break in Sep-Oct; season had only been repairing their damage (market level alone 91.4; old recipe with season 56.6; season added to the new recipe 61.2 vs 44.0). Permutation cannot measure a time feature on future dates, so every importance number in this report comes from refitting without the group.

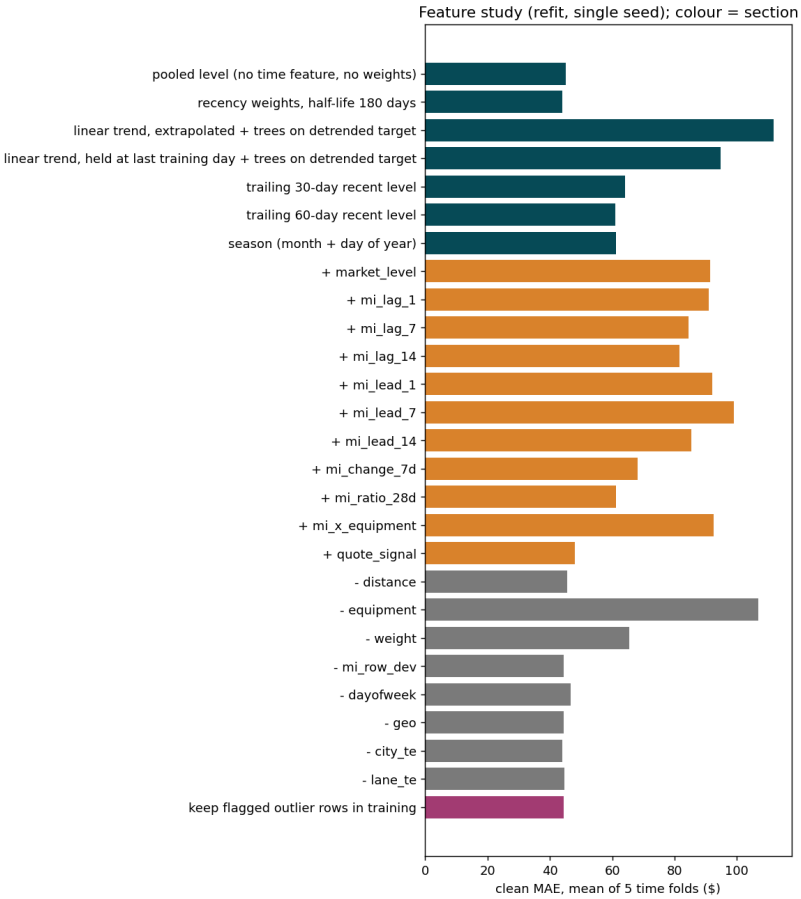


Figure 9: the complete feature study, clean MAE (mean of 5 time folds). Blue = time-level options, orange = added to the shipped recipe, grey = removed from it, purple = training with the spike rows kept.

6. Models compared

6.1 Candidates and results

All candidates predict log(rate per mile) and see the same features and folds. Metrics are means over the 5 time folds (all, clean) and the 3 cold-start folds (unseen-city); all MAPE values and the per-fold results are in Appendix A1-A2.

model	why tried	score	all MAE	clean MAE	clean RMSE	unseen MAE
`blend_lgbm_hierarchical`	**equal average of the two below; their errors differ (trees vs partial pooling)**	**0.648**	**97.9**	**43.5**	**63.2**	**114.3**
`lightgbm+recency`	tuned LightGBM with recency weights, 5 seeds	0.651	98.3	43.9	63.4	115.2
`blend_lgbm_catboost`	average of two boosting libraries	0.651	98.2	43.8	63.4	115.3
`lightgbm`	same, without recency weights	0.659	99.4	45.0	65.1	115.9
`catboost+recency`	second boosting library, ordered boosting, depth 6	0.661	99.1	44.7	64.7	117.5
`hierarchical_shrinkage`	transparent mixed-effects style model; strong for sparse lanes	0.665	99.9	45.6	66.3	116.5
`linear_huber_gam`	spline on log distance + one-hot + Huber; checks how much trees add	0.669	100.2	45.8	66.4	117.9
`lightgbm_v1`	my previous submission's recipe (market level + season, 63 leaves)	0.764	110.9	56.6	82.6	129.7
`lightgbm+season`	reference: does season help the new recipe?	0.776	115.0	60.9	87.1	125.1
`baseline_lane_median`	lane+equipment median rate per mile; the spreadsheet benchmark	1.000	138.8	84.9	123.2	157.9

The lane-median baseline is beaten by every model. Removing the broken market features and season (lightgbm_v1 0.764 → lightgbm 0.659) was the biggest single step; recency weights, then blending with the hierarchical model, added the rest. The top three are close (0.648-0.651); I kept the pre-defined rule and shipped the lowest score. The blend also has the best clean MAE and unseen-city MAE of all candidates.

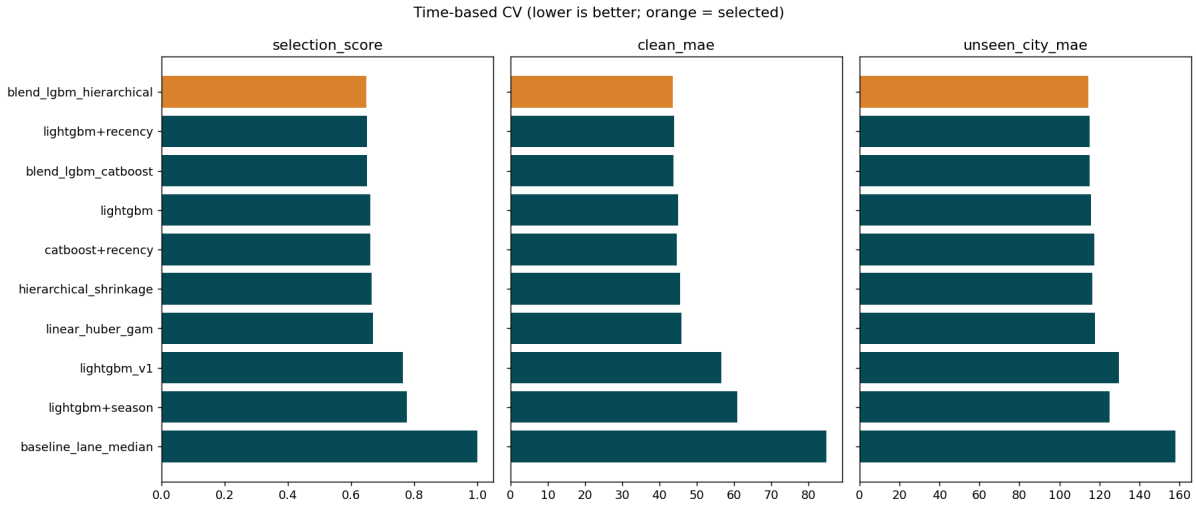


Figure 10: selection score, clean MAE and unseen-city MAE per candidate (orange = selected).

6.2 Tuning

I searched a small grid for LightGBM on the 5 time folds (single seed): num_leaves {7, 15, 31} x n_estimators {600, 1200} x recency half-life {none, 90, 180, 365 days} = 24 settings, scored by the mean of all-row MAE and clean RMSE relative to the baseline. Fixed: learning rate 0.05, min_child_samples 50, row and column subsampling 0.8, L2 1.0, Huber alpha 0.1 in log space. Best: 7 leaves, 600 trees, half-life 180 (score 0.6122; worst 0.6411; all 24 in Appendix A3). Small trees win because lane level, distance and equipment are already carried by the encodings, leaving little interaction to learn. A 90-day half-life does best on rolling_07 but worst on rolling_08, and no weighting the reverse; 180 days is the best compromise across folds.

6.3 Seed averaging

The LightGBM candidates average 5 seeds (42-46) in CV and in the final fit, which removes run-to-run noise from subsampling at the cost of 5x the training time.

6.4 Calibration considered and left off

Because the model estimates a median of log rate, it under-shoots the mean in dollars, and the spikes push the mean up further (a 3x spike adds twice the rate, a 0.3x dip removes 0.7x). I tested two global multipliers fitted on each fold's training rows including spikes:

calibration	all MAE	all RMSE	clean MAE	clean RMSE	unseen MAE	unseen RMSE	mean factor
none	**97.9**	**626.4**	**43.5**	**63.2**	**114.3**	**726.0**	**1.000**
rmse_multiplier	99.0	625.1	44.6	64.0	116.4	724.4	1.009
smearing	99.8	625.0	45.5	65.0	117.4	724.2	1.011

Both lower all-row RMSE by about $1.3-1.5 but raise every MAE and the clean RMSE, because they shift every clean prediction up. No option improves every metric, so I shipped `CALIBRATION = None`; it is one switch in `src/config.py` if Spotter's metric turns out to be all-row RMSE.

7. Final model and validation predictions

blend_lgbm_hierarchical: the average in log space of (a) `lightgbm+recency`, the tuned LightGBM above, and (b) `hierarchical_shrinkage`. Model (b) fits least squares on log distance and its square, equipment, weight and its flags, `mi_row_dev` and weekday, then adds hierarchical empirical-Bayes deviations for region pair, pickup city, delivery city, lane and lane+equipment. Each level's effect is its residual sum divided by (count + k), with k = 50 / 20 / 20 / 10 /

10, fitted by backfitting (3 passes, 3 outer iterations), so a thin lane inherits its cities' and region's level and a busy lane speaks for itself.

The final fit uses all 47,323 clean January-October rows (677 spike rows dropped), calibration factor 1.000. It writes `validation_predictions.csv` with exactly `load_id,predicted_rate` for all 12,000 validation rows (min $195, median $2,041, max $6,888) and fills `data/december_chart_inputs.csv` in place.

Per-fold clean MAE of the final model vs the baseline: rolling_07 59.3 vs 98.0, rolling_08 42.1 vs 82.0, rolling_09 41.7 vs 83.5, rolling_10 35.6 vs 78.7, forward 38.8 vs 82.2. Unseen-city MAE on the three cold-start city sets: 139.9 vs 181.6, 110.2 vs 154.1 and 92.9 vs 138.0. July is the hardest month for every model (it follows the June price peak).

8. The December 2025 chart

The chart file lacks coordinates, market_index and quote_signal. I take coordinates from the fixed city lookup and market_index from each date's national value, which makes `mi_row_dev` = 0; quote_signal is not used. Figure 11 is the chart produced by the provided, unmodified `score.py`.

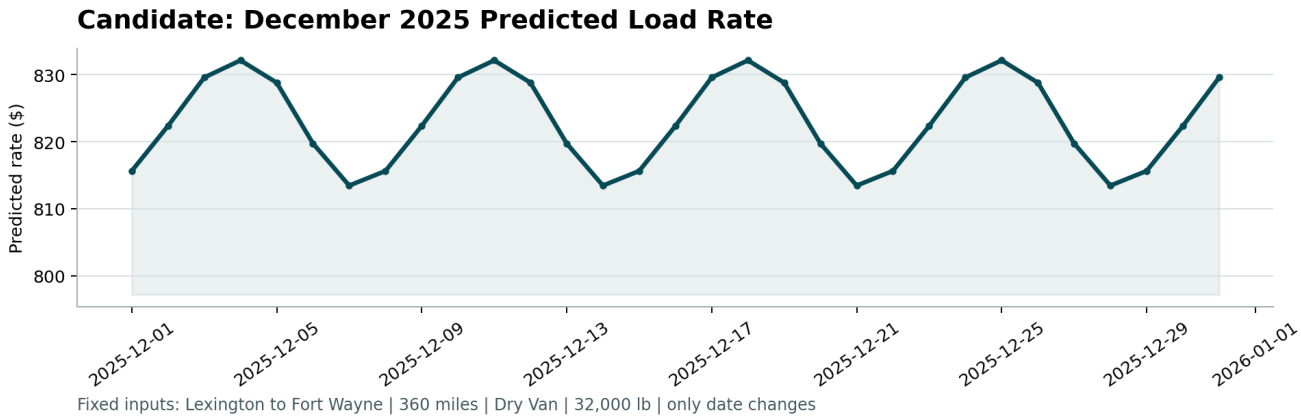


Figure 11: `scorer_results/candidate_december.png` from `score.py`: Lexington → Fort Wayne, 360 mi, Dry Van, 32,000 lb, December 2025.

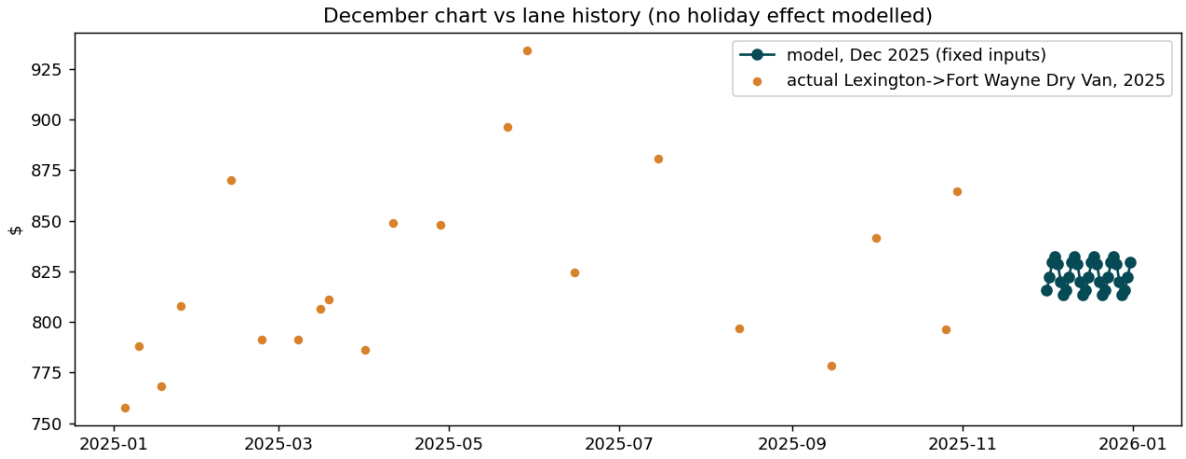


Figure 12: the December line next to the 21 real 2025 loads on the same lane and equipment.

Interpretation. Predictions range from $813.48 to $832.15 (mean $823). Only the date changes, and the model uses no market level, season or trend, so the only moving input is the weekday: the line repeats a 7-day pattern, highest on Thursdays (Dec 4, 11, 18, 25: $832.15) and lowest on Sundays ($813.48), a swing of about 2.3%. That matches the weekday pattern of the market (figure 7) and was learned from rates. The level sits slightly above the lane's 2025 median of $808 (range $758-$934 over 21 loads) and inside the range of its recent loads, consistent with the slightly higher price level of September-October. There is no Thanksgiving-to-Christmas dip or spike, by design: the training data contain no such holiday, so December 25 shows the ordinary Thursday value. My previous version predicted $831-853 (mean $841) for the same inputs.

9. Limitations and next steps

- **Ten months, no holidays.** No yearly seasonality or holiday effect can be learned, and 1-2% month-to-month level shifts are not predictable from the inputs (rolling_07 shows it). With a year or more of history I would add a holiday calendar and a seasonal level and validate them on the previous year's Nov-Dec.
- **Label spikes.** About 1.4% of labels are multiplied by 0.17-5.4x at random; all-row RMSE is set by them. If Spotter knows how they arise (manual entry, a pipeline step), fixing them at the source is worth more than any model change.
- **Unseen cities.** Unseen-city MAE ($114.3) stays above all-row MAE ($97.9). True coordinates and a few loads per new city would help most; a market-area or zip-level hierarchy is the next modelling step.
- **quote_signal and market_index.** quote_signal would be a strong feature if its per-period sign convention were known. The market_index relation broke in August; with more data I would test whether it returns and monitor that relation in production rather than trust it blindly.
- **Point forecasts only.** Quantile models (or conformal intervals on the time folds) would give a price range per load, useful for quoting.

10. Reproducibility

One command, no options: `python main.py` (or `bash run.sh`, which creates a virtual environment, installs the pinned requirements and runs it). Python 3.10-3.13; pinned versions: matplotlib 3.10.3, numpy 2.2.6, pandas 2.2.3, scikit-learn 1.6.1, lightgbm 4.6.0, catboost 1.2.8. Everything is seeded (`SEED = 42` in `src/config.py`). The full run took 14 minutes on my machine with 2 LightGBM threads (`N_JOBS` overrides the thread count). It runs ETL → EDA and outlier diagnostics → tuning and model comparison → final fit and submission files → `score.py` → `reports/summary.md`.

path	content
`main.py`	the pipeline in order (read first)
`src/config.py`	paths, cleaning thresholds, feature groups, folds, model settings, tuning grid, CALIBRATION
`src/data.py`	loading, weight cleaning, national daily market_index, outlier flag, December inputs
`src/eda.py`, `src/outlier_diagnostics.py`	EDA figures, quote_signal and market checks; spike randomness tests
`src/features.py`, `geo.py`, `hierarchy.py`	target, features, out-of-fold encodings; nearest-city borrowing; hierarchical effects
`src/models.py`, `src/cv.py`	candidates and blends; folds, metrics, tuning, comparison, calibration, feature study
`src/predict.py`, `src/report.py`	final fit, output files, December figure, score.py; summary and figures
`validation_predictions.csv`, `scorer_results/`	the submission and the scorer's December chart
`reports/`, `artifacts/`	summary.md, outlier_diagnostics.md, 13 figures; CSV tables behind every number

Appendix: complete results

Everything I tried. 10 candidates in the final model comparison and 14 in my first iteration, 24 entries in total; `lightgbm_v1` in the final comparison re-runs the first iteration's winner (`lightgbm+season`) on the new folds, so 23 distinct model recipes. On top of these: 24 LightGBM tuning settings, 27 refit variants in the final feature study, and 14 refit variants in the first iteration's ablation (10 feature changes, 4 cleaning changes). A1-A6 come from the final run; A7 from the first iteration, whose folds were the same but whose models and baseline-relative score refer to the first version's feature set, so only compare numbers within a table.

A1. Final model comparison, all metrics (means: 5 time folds for all/clean, 3 cold-start folds for unseen-city; $ and %)

model	selection score	all MAE	all RMSE	all MAPE	clean MAE	clean RMSE	clean MAPE	unseen MAE	unseen RMSE	unseen MAPE
blend_lgbm_hierarchical	**0.648**	**97.9**	**626.4**	**4.25%**	**43.5**	**63.2**	**1.83%**	**114.3**	**726.0**	**4.53%**
lightgbm+recency	0.651	98.3	626.1	4.27%	43.9	63.4	1.85%	115.2	725.7	4.57%
blend_lgbm_catboost	0.651	98.2	626.0	4.27%	43.8	63.4	1.85%	115.3	725.2	4.57%
lightgbm	0.659	99.4	627.2	4.31%	45.0	65.1	1.90%	115.9	726.6	4.60%
catboost+recency	0.661	99.1	626.0	4.32%	44.7	64.7	1.90%	117.5	725.0	4.65%
hierarchical_shrinkage	0.665	99.9	627.1	4.33%	45.6	66.3	1.92%	116.5	726.6	4.61%
linear_huber_gam	0.669	100.2	627.0	4.34%	45.8	66.4	1.93%	117.9	727.4	4.66%
lightgbm_v1	0.764	110.9	629.4	4.81%	56.6	82.6	2.39%	129.7	731.8	5.04%
lightgbm+season	0.776	115.0	628.8	5.01%	60.9	87.1	2.57%	125.1	730.5	4.90%
baseline_lane_median	1.000	138.8	637.2	5.97%	84.9	123.2	3.57%	157.9	732.7	6.52%

A2. Per-fold results of the final comparison

roll_MM = rolling fold testing month MM; fwd = forward (Jan-Aug → Sep-Oct); cold_i = cold-start city set i (its all/clean rows also test Sep-Oct).

Clean-row MAE ($)

model	roll_07	roll_08	roll_09	roll_10	forward	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**59.3**	**42.1**	**41.7**	**35.6**	**38.8**	**39.4**	**39.6**	**39.6**
lightgbm+recency	55.5	47.7	42.2	35.2	38.7	39.4	40.0	39.7
blend_lgbm_catboost	55.8	47.2	42.2	35.2	38.8	39.4	40.1	40.9
lightgbm	64.8	38.5	43.1	37.8	40.8	41.3	41.3	41.6
catboost+recency	56.9	47.6	43.1	36.2	39.9	40.9	41.5	43.8
hierarchical_shrinkage	64.8	39.4	43.8	38.5	41.4	42.0	42.1	41.9
linear_huber_gam	64.7	40.6	44.1	38.4	41.5	42.4	42.7	42.2
lightgbm_v1	86.6	36.5	61.6	42.5	55.9	54.7	57.2	55.5
lightgbm+season	118.0	33.5	50.7	51.8	50.4	50.8	50.6	50.4
baseline_lane_median	98.0	82.0	83.5	78.7	82.2	82.0	83.1	82.4

All-row MAE ($)

model	roll_07	roll_08	roll_09	roll_10	forward	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**113.1**	**92.2**	**93.0**	**96.2**	**94.9**	**95.4**	**95.6**	**95.6**
lightgbm+recency	109.5	97.8	93.5	95.8	94.7	95.4	96.0	95.8
blend_lgbm_catboost	109.7	97.3	93.4	95.8	94.8	95.4	96.1	96.8
lightgbm	118.6	88.7	94.4	98.4	96.8	97.3	97.3	97.6
catboost+recency	110.8	97.7	94.4	96.8	95.8	96.8	97.5	99.7
hierarchical_shrinkage	118.6	89.6	95.1	99.0	97.4	98.0	98.1	97.8
linear_huber_gam	118.5	90.7	95.3	99.0	97.5	98.4	98.7	98.2
lightgbm_v1	140.2	86.7	112.6	103.0	111.7	110.5	113.0	111.4
lightgbm+season	171.0	83.8	101.9	112.1	106.3	106.7	106.5	106.3
baseline_lane_median	151.4	131.3	134.5	138.8	137.8	137.6	138.7	137.9

Clean-row RMSE ($)

model	roll_07	roll_08	roll_09	roll_10	forward	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**82.9**	**60.5**	**63.5**	**51.3**	**57.9**	**58.6**	**58.9**	**58.9**
lightgbm+recency	78.2	67.4	63.2	50.7	57.3	58.0	59.1	58.8
blend_lgbm_catboost	78.7	66.7	63.3	50.6	57.5	58.2	59.3	61.8
lightgbm	89.9	55.8	65.4	53.9	60.4	60.9	60.9	61.4
catboost+recency	80.4	67.3	64.8	52.0	59.1	60.4	61.4	75.3
hierarchical_shrinkage	90.2	57.4	66.9	55.5	61.8	62.6	62.8	62.3
linear_huber_gam	89.5	59.1	66.5	55.3	61.4	62.8	63.4	62.5
lightgbm_v1	123.2	52.1	91.9	64.2	81.6	79.8	83.4	81.4
lightgbm+season	158.0	49.1	77.0	78.2	73.0	73.7	73.6	73.3
baseline_lane_median	140.0	119.2	122.8	114.0	120.1	117.9	120.0	118.2

All-row RMSE ($)

model	roll_07	roll_08	roll_09	roll_10	forward	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**630.8**	**613.3**	**612.5**	**645.7**	**629.7**	**629.9**	**630.0**	**629.8**
lightgbm+recency	629.7	613.3	612.2	645.6	629.6	629.8	629.8	629.5
blend_lgbm_catboost	629.7	613.4	612.0	645.5	629.3	629.6	629.6	629.8
lightgbm	632.2	613.1	613.4	646.7	630.7	630.8	630.6	630.5
catboost+recency	629.9	613.6	611.9	645.5	629.3	629.6	629.6	631.2
hierarchical_shrinkage	632.4	613.8	613.1	646.1	630.2	630.3	630.6	630.4
linear_huber_gam	632.1	613.0	613.3	646.2	630.4	630.9	630.8	630.7
lightgbm_v1	632.1	615.9	618.6	644.9	635.3	634.7	635.5	634.8
lightgbm+season	635.3	613.4	616.1	645.6	633.6	633.8	633.9	633.6
baseline_lane_median	644.3	622.6	624.2	654.7	640.3	640.2	640.2	638.9

Clean-row MAPE

model	roll_07	roll_08	roll_09	roll_10	forward	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**2.41%**	**1.78%**	**1.76%**	**1.53%**	**1.65%**	**1.68%**	**1.69%**	**1.69%**
lightgbm+recency	2.26%	2.02%	1.78%	1.52%	1.65%	1.69%	1.71%	1.70%
blend_lgbm_catboost	2.28%	2.00%	1.78%	1.53%	1.65%	1.69%	1.71%	1.80%
lightgbm	2.63%	1.64%	1.82%	1.64%	1.74%	1.77%	1.77%	1.79%
catboost+recency	2.33%	2.03%	1.83%	1.58%	1.71%	1.75%	1.77%	2.00%
hierarchical_shrinkage	2.64%	1.67%	1.85%	1.66%	1.77%	1.80%	1.79%	1.79%
linear_huber_gam	2.64%	1.71%	1.86%	1.65%	1.77%	1.81%	1.82%	1.81%
lightgbm_v1	3.68%	1.60%	2.53%	1.81%	2.33%	2.30%	2.39%	2.31%
lightgbm+season	4.94%	1.43%	2.15%	2.20%	2.15%	2.17%	2.15%	2.14%
baseline_lane_median	4.07%	3.53%	3.48%	3.34%	3.45%	3.51%	3.56%	3.53%

All-row MAPE

model	roll_07	roll_08	roll_09	roll_10	forward	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**4.49%**	**4.25%**	**3.81%**	**4.51%**	**4.18%**	**4.21%**	**4.21%**	**4.21%**
lightgbm+recency	4.36%	4.49%	3.84%	4.51%	4.18%	4.21%	4.24%	4.23%
blend_lgbm_catboost	4.37%	4.48%	3.84%	4.51%	4.18%	4.21%	4.23%	4.32%
lightgbm	4.71%	4.11%	3.87%	4.61%	4.26%	4.29%	4.29%	4.30%
catboost+recency	4.42%	4.52%	3.89%	4.55%	4.24%	4.28%	4.29%	4.52%
hierarchical_shrinkage	4.71%	4.13%	3.90%	4.62%	4.28%	4.31%	4.31%	4.31%
linear_huber_gam	4.71%	4.17%	3.91%	4.62%	4.29%	4.33%	4.33%	4.32%
lightgbm_v1	5.86%	4.01%	4.54%	4.82%	4.81%	4.78%	4.87%	4.80%
lightgbm+season	7.13%	3.88%	4.17%	5.23%	4.63%	4.65%	4.64%	4.63%
baseline_lane_median	6.12%	5.95%	5.52%	6.30%	5.96%	6.02%	6.07%	6.03%

Clean-row bias (mean signed % error)

model	roll_07	roll_08	roll_09	roll_10	forward	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**-1.8**	**+1.4**	**-0.4**	**-0.6**	**-0.5**	**-0.5**	**-0.4**	**-0.5**
lightgbm+recency	-1.5	+1.7	-0.2	-0.4	-0.3	-0.3	-0.2	-0.3
blend_lgbm_catboost	-1.5	+1.7	-0.2	-0.4	-0.3	-0.3	-0.2	-0.3
lightgbm	-2.1	+1.0	-0.7	-0.8	-0.8	-0.8	-0.6	-0.8
catboost+recency	-1.5	+1.7	-0.2	-0.4	-0.3	-0.3	-0.3	-0.2
hierarchical_shrinkage	-2.1	+1.0	-0.7	-0.8	-0.8	-0.7	-0.7	-0.8
linear_huber_gam	-2.1	+1.1	-0.6	-0.7	-0.7	-0.8	-0.7	-0.7
lightgbm_v1	+3.5	-1.0	-2.2	+0.9	-2.0	-2.0	-2.1	-2.0
lightgbm+season	+4.9	+0.4	-1.6	+1.4	-1.7	-1.7	-1.6	-1.6
baseline_lane_median	-2.1	+1.2	-0.6	-0.8	-0.8	-0.8	-0.6	-0.9

Unseen-city MAE ($)

model	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**139.9**	**110.2**	**92.9**
lightgbm+recency	140.0	111.3	94.2
blend_lgbm_catboost	140.1	111.4	94.4
lightgbm	141.3	111.2	95.0
catboost+recency	142.4	113.5	96.5
hierarchical_shrinkage	142.6	113.0	93.9
linear_huber_gam	143.3	115.0	95.5
lightgbm_v1	154.0	130.0	105.1
lightgbm+season	151.5	121.2	102.5
baseline_lane_median	181.6	154.1	138.0

Unseen-city RMSE ($)

model	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**912.3**	**693.8**	**571.8**
lightgbm+recency	912.5	693.5	571.3
blend_lgbm_catboost	911.9	692.9	570.8
lightgbm	913.8	694.0	572.0
catboost+recency	911.7	692.6	570.6
hierarchical_shrinkage	912.5	694.7	572.6
linear_huber_gam	914.0	695.7	572.4
lightgbm_v1	917.7	700.4	577.3
lightgbm+season	917.5	698.3	575.8
baseline_lane_median	918.9	701.7	577.6

Unseen-city MAPE

model	cold_1	cold_2	cold_3
blend_lgbm_hierarchical	**4.40%**	**4.76%**	**4.42%**
lightgbm+recency	4.42%	4.83%	4.47%
blend_lgbm_catboost	4.42%	4.82%	4.47%
lightgbm	4.45%	4.81%	4.53%
catboost+recency	4.49%	4.90%	4.56%
hierarchical_shrinkage	4.49%	4.86%	4.47%
linear_huber_gam	4.53%	4.93%	4.53%
lightgbm_v1	4.87%	5.43%	4.83%
lightgbm+season	4.79%	5.14%	4.78%
baseline_lane_median	6.28%	6.79%	6.50%

A3. LightGBM tuning grid, all 24 settings (5 time folds, single seed; sorted by tuning score)

score = mean(all MAE, clean RMSE) relative to the baseline; fwd / r07-r10 = clean MAE per fold.

#	leaves	trees	half-life	score	all MAE	clean MAE	all RMSE	clean RMSE	fwd	r07	r08	r09	r10	r07 bias %
1	**7**	**600**	**180**	**0.6122**	**98.3**	**44.0**	**626.1**	**63.5**	**38.8**	**55.4**	**47.7**	**42.4**	**35.4**	**-1.5**
2	7	1200	180	0.6127	98.4	44.0	626.0	63.6	38.9	55.9	47.6	42.4	35.4	-1.5
3	15	600	180	0.6134	98.5	44.1	626.1	63.7	39.0	56.2	47.5	42.3	35.5	-1.5
4	7	600	365	0.6154	98.6	44.3	626.6	64.1	39.6	59.9	42.8	42.5	36.4	-1.8
5	15	600	365	0.6158	98.7	44.3	626.5	64.1	39.6	60.2	42.8	42.4	36.5	-1.8
6	7	1200	365	0.6161	98.7	44.3	626.5	64.2	39.6	60.2	43.0	42.5	36.4	-1.8
7	7	600	90	0.6171	99.1	44.8	625.6	64.1	38.8	49.0	58.0	43.0	34.9	-0.8
8	15	1200	180	0.6172	98.8	44.5	626.1	64.3	39.4	57.3	47.4	42.6	35.8	-1.5
9	7	1200	90	0.6173	99.1	44.7	625.5	64.1	38.7	49.6	57.6	43.0	34.8	-0.9
10	31	600	180	0.6184	98.9	44.6	626.0	64.5	39.6	57.3	47.2	42.8	36.1	-1.5
11	15	600	90	0.6189	99.2	44.9	625.6	64.4	39.0	49.9	57.4	43.0	35.2	-0.9
12	15	1200	365	0.6207	99.1	44.8	626.5	64.9	40.1	61.0	43.1	42.8	36.9	-1.8
13	31	600	365	0.6217	99.3	44.9	626.5	65.1	40.4	61.0	43.1	43.0	37.1	-1.8
14	15	1200	90	0.6224	99.5	45.2	625.6	65.0	39.3	51.3	56.6	43.3	35.4	-1.0
15	7	1200	none	0.6226	99.4	45.0	627.1	65.2	40.9	64.7	38.7	43.2	37.8	-2.1
16	31	600	90	0.6229	99.6	45.3	625.5	65.1	39.7	51.3	56.2	43.7	35.4	-1.0
17	7	600	none	0.6230	99.5	45.1	627.3	65.2	41.0	64.6	38.7	43.3	37.9	-2.1
18	15	600	none	0.6258	99.7	45.4	627.1	65.6	41.2	65.0	39.3	43.4	38.0	-2.1
19	31	1200	180	0.6275	99.8	45.5	626.2	66.0	40.6	59.0	47.0	43.8	37.1	-1.6
20	31	1200	90	0.6290	100.1	45.8	625.7	66.1	40.4	53.6	54.6	44.2	36.1	-1.1
21	15	1200	none	0.6299	100.1	45.8	627.1	66.3	41.5	65.2	39.9	43.9	38.4	-2.1
22	31	1200	365	0.6307	100.2	45.8	626.6	66.5	41.4	62.3	43.6	43.8	38.0	-1.9
23	31	600	none	0.6309	100.2	45.9	627.0	66.5	41.7	65.4	39.9	43.8	38.7	-2.1
24	31	1200	none	0.6411	101.3	47.0	627.0	68.1	42.8	66.2	41.1	45.0	39.6	-2.1

A4. Feature study (refit on 5 time folds, single seed)

Time-level options (fwd / r07-r10 = clean MAE per fold; r07 bias = mean signed % error on rolling_07)

variant	clean MAE	all MAE	clean RMSE	all RMSE	fwd	r07	r08	r09	r10	r07 bias %
pooled level (no time feature, no weights)	45.1	99.5	65.2	627.3	41.0	64.6	38.7	43.3	37.9	-2.1
recency weights, half-life 180 days	44.0	98.3	63.5	626.1	38.8	55.4	47.7	42.4	35.4	-1.5
linear trend, extrapolated + trees on detrended target	111.9	165.2	141.6	632.5	98.5	123.9	175.1	91.6	70.3	+5.3
linear trend, held at last training day + trees on detrended target	94.7	148.2	122.1	629.4	77.0	96.8	154.8	82.5	62.3	+4.1
trailing 30-day recent level	64.2	118.2	88.9	628.7	44.7	120.0	76.9	44.7	34.5	+4.9
trailing 60-day recent level	60.9	115.0	85.3	627.0	44.3	119.4	54.8	50.8	35.4	+4.9
season (month + day of year)	61.2	115.3	87.4	629.0	50.3	119.0	34.0	50.6	52.0	+4.9

Added to the shipped recipe (shipped: clean MAE 44.0, all MAE 98.3, all RMSE 626.1)

variant	clean MAE	all MAE	clean RMSE	all RMSE	fwd	r07	r08	r09	r10
+ market_level	91.4	145.1	118.4	640.7	114.4	48.5	74.8	140.7	78.6
+ mi_lag_1	91.0	144.7	118.2	640.4	117.4	46.1	72.9	142.5	75.8
+ mi_lag_7	84.5	138.3	111.0	638.5	123.0	44.2	56.2	132.4	66.7
+ mi_lag_14	81.7	135.5	108.7	637.7	120.4	43.3	43.3	116.6	84.8
+ mi_lead_1	92.1	145.9	119.7	641.0	113.3	52.8	76.7	143.5	74.3
+ mi_lead_7	99.1	152.7	127.7	642.4	107.5	92.7	90.7	136.3	68.3
+ mi_lead_14	85.4	139.2	114.3	638.5	74.4	112.0	102.0	64.2	74.3
+ mi_change_7d	68.2	122.2	97.3	629.6	60.5	101.7	59.7	47.6	71.4
+ mi_ratio_28d	61.2	115.4	90.4	629.7	52.7	97.1	53.9	50.1	52.3
+ mi_x_equipment	92.6	146.3	121.5	641.5	116.5	49.8	74.6	145.7	76.4
+ quote_signal	48.0	102.2	70.9	626.6	35.1	49.2	85.2	35.3	35.1

Removed from the shipped recipe, and the cleaning check

variant	clean MAE	all MAE	clean RMSE	all RMSE	fwd	r07	r08	r09	r10
- distance	45.5	99.8	65.3	626.1	40.4	56.6	49.4	43.5	37.4
- equipment	106.9	160.1	149.7	639.1	108.6	101.4	107.1	119.4	98.0
- weight	65.5	119.7	92.3	630.2	62.9	72.4	66.3	64.4	61.5
- mi_row_dev	44.5	98.8	64.2	626.2	39.3	56.3	48.1	42.8	35.8
- dayofweek	46.7	101.1	67.7	627.0	41.5	58.6	50.4	45.0	38.1
- geo	44.3	98.7	64.0	626.2	39.3	55.8	48.2	42.7	35.7
- city_te	44.0	98.4	63.5	626.0	38.9	55.3	47.9	42.4	35.5
- lane_te	44.6	99.0	64.5	626.3	39.7	55.9	47.8	43.0	36.5
keep flagged outlier rows in training	44.5	98.9	64.1	626.3	39.4	55.9	48.1	43.0	36.1

A5. quote_signal by month and price level by month

month	corr(quote, rpm)	slope	intercept	resid sd	rows	corr vs lane median rpm
2025-01 (train)	+0.996	+1.000	-0.001	0.022	4,837	+0.952
2025-02 (train)	+0.996	+0.999	0.001	0.022	4,280	+0.957
2025-03 (train)	+0.997	+1.001	-0.002	0.022	4,965	+0.962
2025-04 (train)	-0.993	-1.002	4.154	0.030	4,753	-0.958
2025-05 (train)	-0.994	-1.000	4.150	0.029	4,851	-0.961
2025-06 (train)	+0.997	+0.999	0.002	0.022	4,720	+0.958
2025-07 (train)	-0.994	-0.999	4.147	0.030	4,847	-0.958
2025-08 (train)	-0.020	-0.017	2.090	0.223	4,691	-0.026
2025-09 (train)	+0.997	+1.000	0.000	0.022	4,606	+0.966
2025-10 (train)	-0.994	-0.997	4.144	0.030	4,773	-0.964
2025-11 (validation)	+0.002	-	-	-	4,915	+0.002
2025-12 (validation)	-0.018	-	-	-	5,218	-0.018

month 2025	1	2	3	4	5	6	7	8	9	10
price level (% vs lane mean)	-4.1	-3.0	-0.4	+0.1	+1.9	+3.6	+1.5	-0.8	+0.4	+0.7
market_index (monthly mean)	0.93	1.00	1.07	1.21	1.30	1.28	1.20	0.98	0.89	0.96

A6. Outlier diagnostics (`reports/outlier_diagnostics.md`)

Size of the spikes (ratio to lane reference)

	high spikes (ratio)	low dips (ratio)	low dips (1 / ratio)
min	2.077	0.168	2.120
p10	2.443	0.198	2.526
p25	2.864	0.222	2.841
median	3.344	0.283	3.529
p75	4.226	0.352	4.507
p90	4.675	0.396	5.060
max	5.441	0.472	5.954

Flag rate by factor. min/max rate = lowest/highest flagged share among groups with ≥ 100 loads; ci_excl = groups whose 95% Wilson interval excludes 1.41%; p-values: chi-square of flag vs group, dir3 = normal/high/low, high_vs_low = direction within flagged rows, size_* = Kruskal-Wallis of |log ratio|; _holm = Holm-corrected over the table.

	groups	min_rate	max_rate	groups_ci_excl_overall	cramers_v	chi2_p	chi2_p_holm	dir3_p_holm	high_vs_low_p_holm	size_high_p_holm	size_low_p_holm
equipment	3	1.35%	1.51%	0	0.006	0.443	1	1	1	1	1
distance band	7	1.21%	2.01%	0	0.014	0.163	1	1	1	1	1
pickup city	64	0.00%	3.26%	4	0.034	0.76	1	1	1	1	1
delivery city	64	0.43%	2.66%	4	0.041	0.0507	1	1	1	1	1
pickup region	4	1.08%	1.46%	0	0.010	0.205	1	1	1	1	1
delivery region	4	1.12%	1.49%	0	0.010	0.168	1	1	1	1	1
region pair	16	0.76%	1.63%	0	0.016	0.626	1	1	1	1	1
lane size (loads)	3	1.34%	1.45%	0	0.004	0.714	1	1	1	1	1
reference used	2	1.34%	1.42%	0	0.002	0.643	1	1	1	1	1
month	10	1.26%	1.65%	0	0.011	0.768	1	1	1	1	1
day of week	7	1.23%	1.59%	0	0.010	0.6	1	1	1	1	1
day of month	31	0.99%	2.35%	1	0.023	0.724	1	1	1	1	1
within 3 days of a holiday	2	1.41%	1.41%	0	0.000	1	1	1	1	1	1
nearest holiday (in window)	9	0.95%	1.82%	0	0.010	0.776	1	1	1	1	1
market_index quintile	5	1.24%	1.65%	0	0.011	0.186	1	1	1	1	1
market_index daily-change quintile	5	1.27%	1.65%	0	0.011	0.199	1	1	1	1	1
market_index weekly-deviation quintile	5	1.30%	1.50%	0	0.006	0.774	1	1	1	0.892	1
weight quintile	5	1.25%	1.55%	0	0.011	0.255	1	1	1	1	0.559
weight missing	2	1.00%	1.41%	0	0.002	0.719	1	1	1	-	-
weight negative	2	1.40%	3.08%	1	0.010	0.0292	1	1	1	1	-
weight capped	2	1.33%	1.41%	0	0.001	0.905	1	1	1	1	1
market_index missing	2	1.41%	1.60%	0	0.000	0.921	1	1	1	-	-
quote_signal quintile	5	1.30%	1.58%	0	0.008	0.489	1	1	1	1	1
load_id order quintile	5	1.29%	1.51%	0	0.006	0.738	1	1	1	1	1

Numeric columns, flagged vs normal (auc_single = Mann-Whitney AUC of the one column)

	median_flagged	median_normal	auc_single	mw_p	mw_p_holm	high_vs_low_p_holm	spearman_size_p_holm
distance	874.4	954.1	0.474	0.0193	0.309	1	1
weight_clean	3.097e+04	3.149e+04	0.484	0.163	1	1	1
market_index_clean	1.048	1.056	0.494	0.57	1	1	0.127
mi_day	1.043	1.052	0.492	0.468	1	1	0.153
mi_change	0.01482	0.01176	0.517	0.124	1	0.433	1
mi_week_dev	-0.008818	-0.009323	0.515	0.193	1	1	1
quote_signal	2.04	2.056	0.493	0.557	1	1	1
pickup_lat	35.3	35.29	0.509	0.408	1	1	1
pickup_lon	-87.53	-88.09	0.509	0.406	1	1	1
delivery_lat	35.29	35.29	0.500	0.989	1	1	1
delivery_lon	-87.53	-87.53	0.512	0.276	1	1	1
lane_n	6	6	0.500	0.967	1	1	1
days_to_holiday	10	11	0.484	0.144	1	1	1
dayofmonth	16	16	0.510	0.389	1	1	1
load_seq	2.41e+04	2.4e+04	0.502	0.845	1	1	1
lane_prior_flag_rate	0.01175	0.01175	0.505	0.662	1	1	1

Can a model predict the flag? Train on months < M, test on M; 95% CI from 1,000 bootstrap resamples. First: flagged vs normal; second: high spike vs low dip among flagged rows.

	pooled AUC	95% CI low	95% CI high	mean fold AUC	positives	rows
logistic regression	0.508	0.482	0.534	0.506	468	33709
LightGBM classifier	0.513	0.489	0.537	0.505	468	33709

	pooled AUC	95% CI low	95% CI high	mean fold AUC	high spikes	flagged rows
logistic regression	0.535	0.482	0.584	0.543	236	468
LightGBM classifier	0.515	0.462	0.569	0.519	236	468

Most extreme lanes (none significant after Holm). Time clustering: daily dispersion 0.98 (p = 0.569); lag-1 autocorrelation 0.092 (p = 0.11). Lane clustering: dispersion 1.12 vs a simulated null median 1.00, 95th percentile 1.14, simulation p = 0.075. P(flag | previous load on lane flagged) = 2.50% vs 1.40% (Fisher p = 0.0572).

	loads	flagged	p	p_holm	
Greensboro>Baton Rouge	Dry Van	11	3	0.000425	0.439
Los Angeles>Mobile	Dry Van	13	3	0.000722	0.745
Tucson>Phoenix	Dry Van	13	3	0.000722	0.745
Oklahoma City>Mobile	Dry Van	22	3	0.00353	1
Chattanooga>Grand Rapids	Dry Van	10	2	0.0083	1

A7. First iteration (earlier version of the pipeline)

Same fold design and selection-score definition, but every LightGBM / CatBoost model there used the market_index level features; the selected model was `lightgbm+season`. MAPE in %. In the first iteration only `lightgbm+season` and `lightgbm+quote_signal` beat the lane-median baseline, which is what led me to the market_index analysis of section 5.2.

Model comparison (14 candidates)

model	selection_score	all_rmse	all_mae	all_mape	clean_rmse	clean_mae	clean_mape	unseen_city_rmse	unseen_city_mae	unseen_city_mape
lightgbm+season	**0.763**	**629.2**	**110.6**	**4.79%**	**82.3**	**56.4**	**2.37%**	**731.6**	**130.1**	**5.07%**
lightgbm+quote_signal	0.926	635.4	131.0	5.75%	105.6	77.1	3.43%	737.9	154.2	5.90%
hierarchical_shrinkage	0.997	637.9	135.1	5.80%	105.8	81.2	3.47%	746.2	183.2	7.12%
baseline_lane_median	1.000	637.2	138.8	5.97%	123.2	84.9	3.57%	732.7	157.9	6.52%
blend_hier_shrinkage_lgbm	1.030	639.2	139.8	5.96%	111.2	86.0	3.63%	747.3	186.4	7.18%
lightgbm+hier_te+city_masking	1.076	641.0	146.0	6.18%	119.9	92.3	3.86%	748.9	190.0	7.27%
lightgbm+hier_te	1.079	641.2	146.6	6.20%	120.6	92.9	3.88%	749.0	189.7	7.25%
lightgbm_no_city_te	1.085	641.4	146.9	6.20%	121.3	93.2	3.89%	749.1	191.2	7.33%
lightgbm+city_masking	1.087	641.2	146.9	6.21%	121.0	93.2	3.89%	749.6	192.6	7.41%
lightgbm	1.088	641.3	146.7	6.20%	120.8	92.9	3.88%	750.4	193.5	7.41%
blend_hier_masked_lgbm_catboost	1.094	641.6	147.3	6.23%	123.0	93.7	3.91%	750.6	192.7	7.36%
blend_lgbm_catboost	1.100	641.7	147.7	6.24%	123.5	94.0	3.93%	751.3	194.5	7.43%
linear_huber_gam	1.108	641.7	148.4	6.34%	121.8	94.7	4.04%	751.4	199.8	7.75%
catboost	1.139	643.5	150.1	6.32%	133.3	96.4	4.02%	759.1	198.1	7.53%

MAE per fold ($), cold-start folds

model	cold_1 all	cold_1 clean	cold_2 all	cold_2 clean	cold_3 all	cold_3 clean
lightgbm+season	**110.8**	**55.0**	**112.3**	**56.5**	**111.3**	**55.5**
lightgbm+quote_signal	133.0	77.5	134.2	78.7	134.7	79.2
hierarchical_shrinkage	165.5	110.5	164.8	109.7	165.8	110.7
baseline_lane_median	137.6	82.0	138.7	83.1	137.9	82.4
blend_hier_shrinkage_lgbm	167.9	112.9	167.9	112.9	168.2	113.2
lightgbm+hier_te+city_masking	170.4	115.5	171.3	116.3	170.9	115.9
lightgbm+hier_te	170.9	116.0	170.3	115.3	171.5	116.5
lightgbm_no_city_te	171.4	116.4	171.4	116.4	171.5	116.5
lightgbm+city_masking	171.7	116.8	171.7	116.7	170.7	115.8
lightgbm	170.4	115.5	172.0	117.1	172.2	117.3
blend_hier_masked_lgbm_catboost	171.1	116.1	175.7	120.8	171.6	116.6
blend_lgbm_catboost	171.1	116.1	176.1	121.2	172.3	117.3
linear_huber_gam	177.6	122.7	176.8	121.9	177.0	122.1
catboost	172.0	117.1	185.2	130.5	172.8	117.8

MAE per fold ($), time folds

model	forward all	forward clean	roll_07 all	roll_07 clean	roll_08 all	roll_08 clean	roll_09 all	roll_09 clean	roll_10 all	roll_10 clean
lightgbm+season	**111.6**	**55.8**	**139.4**	**85.8**	**87.6**	**37.5**	**112.1**	**61.0**	**102.3**	**41.8**
lightgbm+quote_signal	133.3	77.8	95.2	41.2	162.3	113.4	134.3	83.6	129.7	69.7
hierarchical_shrinkage	166.7	111.7	93.1	38.9	110.5	60.7	172.9	122.6	132.3	72.2
baseline_lane_median	137.8	82.2	151.4	98.0	131.3	82.0	134.5	83.5	138.8	78.7
blend_hier_shrinkage_lgbm	168.5	113.5	94.4	40.2	116.4	66.7	181.8	131.6	138.0	78.0
lightgbm+hier_te+city_masking	170.5	115.5	101.3	47.2	123.2	73.6	190.7	140.7	144.3	84.4
lightgbm+hier_te	171.4	116.4	101.6	47.5	123.2	73.7	191.4	141.4	145.4	85.6
lightgbm_no_city_te	171.4	116.4	102.3	48.2	122.9	73.3	191.6	141.5	146.3	86.4
lightgbm+city_masking	171.9	116.9	101.2	47.1	123.0	73.4	192.3	142.3	145.9	86.0
lightgbm	171.5	116.5	101.3	47.2	123.2	73.6	191.6	141.6	145.8	85.9
blend_hier_masked_lgbm_catboost	171.6	116.6	103.7	49.6	124.3	74.7	191.4	141.4	145.8	85.9
blend_lgbm_catboost	172.1	117.1	103.8	49.7	124.3	74.7	191.9	141.9	146.5	86.7
linear_huber_gam	176.9	122.0	100.9	46.8	122.2	72.5	192.6	142.5	149.3	89.5
catboost	173.8	118.8	108.6	54.6	127.2	77.7	193.3	143.3	147.6	87.8

Refit ablation of `lightgbm+season`: deltas vs the full setup (forward fold for all/clean, cold_start_1 for unseen-city; positive = error rose). Full setup: all RMSE 635.0, all MAE 111.6, all MAPE 4.80%, clean RMSE 81.5, clean MAE 55.8, clean MAPE 2.32%, unseen RMSE 918.0, unseen MAE 155.8, unseen MAPE 4.94%.

change	all_rmse	all_mae	all_mape	clean_rmse	clean_mae	clean_mape	unseen_city_rmse	unseen_city_mae	unseen_city_mape
drop distance	-0.3	-1.5	-0.03%	-2.5	-1.5	-0.04%	-0.2	-1.0	+0.01%
drop equipment	+14.3	+51.2	+2.03%	+80.7	+52.2	+2.08%	+10.1	+69.7	+2.69%
drop weight	+4.9	+20.9	+0.86%	+28.5	+21.0	+0.86%	+1.3	+20.2	+0.79%
drop market_index	-1.4	-4.4	-0.18%	-6.6	-4.5	-0.19%	-1.3	-3.7	-0.10%
drop dayofweek	+0.1	-0.2	-0.01%	-0.7	-0.3	-0.01%	-0.4	+0.3	-0.02%
drop geo	+0.5	+1.0	+0.04%	+1.5	+1.0	+0.04%	+0.4	+2.2	+0.08%
drop city_te	+0.2	-0.4	-0.01%	-0.4	-0.4	-0.01%	-0.8	-2.3	-0.10%
drop lane_te	+0.4	+0.1	-0.00%	+0.3	+0.1	-0.00%	+0.5	+2.7	+0.09%
drop season	+17.2	+59.9	+2.44%	+69.3	+60.7	+2.54%	+15.6	+61.3	+2.22%
add quote_signal	-0.4	-2.5	-0.22%	-2.4	-2.5	-0.23%	-0.2	-1.3	-0.22%

Cleaning ablation (forward fold, deltas vs full cleaning: all RMSE 635.0, all MAE 111.6, clean MAE 55.8)

change	all_rmse	all_mae	all_mape	clean_rmse	clean_mae	clean_mape
keep outliers in training	-0.3	-0.2	+0.01%	-0.2	-0.2	+0.01%
keep outliers + squared-error loss	+7.1	+30.6	+1.47%	+46.6	+31.1	+1.47%
raw weight (no abs/fill/flags)	-0.1	-0.3	-0.02%	-0.1	-0.3	-0.02%
raw market_index (no fill/daily features)	-0.0	-1.6	-0.08%	-2.0	-1.7	-0.08%

Permutation importance (forward fold, no refit; error increase) and **LightGBM contributions** (mean |contribution| to log rate per mile). These are the numbers that showed the season contradiction discussed in section 5.2.

group	all_mae_increase	clean_rmse_increase
distance	+168.4	+277.9
equipment	+93.5	+138.3
weight	+30.1	+42.4
market_index	+4.6	+6.4
lane_te	+3.5	+5.5
city_te	+4.6	+4.6
season	+0.0	+0.0
dayofweek	-0.0	-0.0
geo	+1.5	-1.2

group	mean abs contribution
distance	0.0714
equipment	0.0504
market_index	0.0295
weight	0.0218
season	0.0173
geo	0.0162
city_te	0.0110
lane_te	0.0077
dayofweek	0.0001